

Proyecto: Sistema de Encuestas "Espagueti Fase 3 y4 "

Corporación Universitaria del Huila – CORHUILA

Asignatura: Arquitectura de Software

Maria Fernanda Robayo Laguna

Laura Sofia Cangrejo Perafan

Docente: Luis Ángel Vargas Narváez

Fecha: lunes, 23 de febrero de 2026

Tabla de contenido

1. Introducción.....	4
2. Objetivo de la Fase.....	4
3. Actividad 3.1 – Propuesta de Patrones de Diseño	5
3.1 Layered Architecture (MVC)	5
3.2 Repository Pattern (Eliminar SQL Directo).....	8
3.3 DTO Pattern (Data Transfer Object)	14
3.4 Externalized Configuration (Eliminación de Configuración Hardcodeada).....	21
3.5 Global Exception Handler (Manejo Centralizado de Errores).....	26
3.6 Service Layer en Angular	32
4. Actividad 3.2 – Propuesta de Principios Clean Code	43
4.1 Nombres Significativos.....	43
4.2 Funciones Pequeñas y Responsabilidad Única	45
4.3 DRY (Don't Repeat Yourself)	47
4.4 Manejo de Errores – Evitar return null	49
4.5 Comentarios y Código Autoexplicativo	50
5.1 Identificación de Bounded Contexts.....	53
5.2 Propuesta de Microservicios.....	53
5.3.1 survey-service	53
5.3.2 voting-service.....	54
5.3.3 api-gateway.....	55

5.4 Arquitectura Propuesta	56
5.5 Componentes de Infraestructura Adicionales	57
ADR-001 · Adoptar arquitectura de microservicios	59
ADR-002 · Usar Spring Data JPA en lugar de JDBC manual	60
Conclusión General	61

1. Introducción

En las Fases 1 y 2 se realizó un análisis arquitectónico del sistema **“Espaguetti de Encuestas”**, identificando múltiples anti-patrones en el backend (Spring Boot) y frontend (Angular).

El análisis permitió detectar:

- Concentración de responsabilidades en el controlador.
- Uso de SQL concatenado vulnerable a SQL Injection.
- Uso de estructuras genéricas (Map, any) sin contratos tipados.
- Configuración hardcodeada.
- Manejo de errores deficiente.
- Alto acoplamiento entre componentes.
- Ausencia de separación por capas.

Estos hallazgos evidencian que el sistema actual funciona como un monolito sin arquitectura definida, con bajo nivel de mantenibilidad y escalabilidad limitada.

La presente fase tiene como objetivo proponer un rediseño arquitectónico fundamentado en patrones reconocidos, principios Clean Code y una posible evolución hacia microservicios.

2. Objetivo de la Fase

El propósito de esta fase es:

1. Proponer soluciones a los anti-patrones identificados.
2. Aplicar patrones de diseño estructurales y arquitectónicos.
3. Mejorar la calidad del código mediante Clean Code.
4. Diseñar una arquitectura orientada a microservicios.
5. Documentar decisiones arquitectónicas mediante ADR.

6. Representar el sistema usando modelo C4.

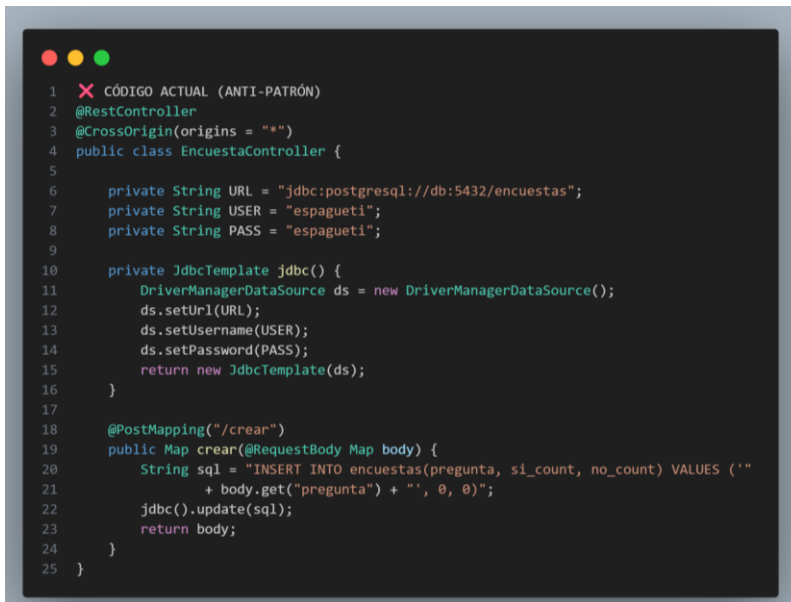
3. Actividad 3.1 – Propuesta de Patrones de Diseño

Para cada anti-patrón identificado en Fase 2, se propone la aplicación de un patrón reconocido.

3.1 Layered Architecture (MVC)

En el sistema actual, el archivo EncuestaController.java concentra múltiples responsabilidades dentro de una sola clase. El controlador no solo gestiona las peticiones HTTP, sino que también realiza validaciones, construye consultas SQL, ejecuta operaciones de base de datos y maneja la configuración de conexión.

Desde el punto de vista arquitectónico, esta situación representa un anti-patrón conocido como “God Controller”, donde una sola clase asume responsabilidades que deberían estar separadas.



```
1  ✖ CÓDIGO ACTUAL (ANTI-PATRÓN)
2  @RestController
3  @CrossOrigin(origins = "**")
4  public class EncuestaController {
5
6      private String URL = "jdbc:postgresql://db:5432/encuestas";
7      private String USER = "espagueti";
8      private String PASS = "espagueti";
9
10     private JdbcTemplate jdbc() {
11         DriverManagerDataSource ds = new DriverManagerDataSource();
12         ds.setUrl(URL);
13         ds.setUsername(USER);
14         ds.setPassword(PASS);
15         return new JdbcTemplate(ds);
16     }
17
18     @PostMapping("/crear")
19     public Map crear(@RequestBody Map body) {
20         String sql = "INSERT INTO encuestas(pregunta, si_count, no_count) VALUES ('"
21             + body.get("pregunta") + "', 0, 0)";
22         jdbc().update(sql);
23         return body;
24     }
25 }
```

Ilustración 1-Código actual: controlador “todo en uno” sin separación por capas.

Problemas Detectados

- El controlador gestiona configuración de base de datos.
- Se instancia manualmente el DataSource.

- La lógica de negocio está mezclada con la capa web.
- No existe separación por capas.
- Se viola el principio SRP (Single Responsibility Principle).

Código Propuesto (Arquitectura por Capas)

A screenshot of a code editor with a dark background and light-colored text. The code is a Java class named `EncuestaController` that implements a REST controller. It includes annotations for `@RestController` and `@CrossOrigin`. The class contains private fields for database connection details (URL, USER, PASS) and a private `JdbcTemplate` instance. A `@PostMapping("/crear")` method is shown, which takes a `RequestBody Map` and executes an SQL insert statement. The code is numbered from 1 to 25.

```
1 Controller
2 @RestController
3 @CrossOrigin(origins = "")
4 public class EncuestaController {
5
6     private String URL = "jdbc:postgresql://db:5432/encuestas";
7     private String USER = "espaguetti";
8     private String PASS = "espaguetti";
9
10    private JdbcTemplate jdbc() {
11        DriverManagerDataSource ds = new DriverManagerDataSource();
12        ds.setUrl(URL);
13        ds.setUsername(USER);
14        ds.setPassword(PASS);
15        return new JdbcTemplate(ds);
16    }
17
18    @PostMapping("/crear")
19    public Map crear(@RequestBody Map body) {
20        String sql = "INSERT INTO encuestas(pregunta, si_count, no_count) VALUES ('"
21            + body.get("pregunta") + "', 0, 0)";
22        jdbc().update(sql);
23        return body;
24    }
25 }
```

Ilustración 2-Propuesta de refactorización del controlador bajo el modelo de arquitectura por capas.

En esta versión se busca separar las responsabilidades del sistema, evitando que el controlador gestione directamente la conexión a base de datos y la ejecución de sentencias SQL. Este enfoque mejora la mantenibilidad, reduce el acoplamiento y permite una mejor organización del código conforme a los principios de diseño limpio y responsabilidad única (SRP).

Service

```
1 Service
2 @Service
3 public class SurveyService {
4
5     private final SurveyRepository surveyRepository;
6
7     public SurveyService(SurveyRepository surveyRepository) {
8         this.surveyRepository = surveyRepository;
9     }
10
11     public SurveyDTO create(CreateSurveyRequest request) {
12         Survey survey = new Survey();
13         survey.setPregunta(request.pregunta());
14         survey.setSiCount(0);
15         survey.setNoCount(0);
16
17         return mapToDTO(surveyRepository.save(survey));
18     }
19 }
```

Ilustración 3-Implementación de la capa de servicio (Service) en la arquitectura por capas.

En esta sección se concentra la lógica de negocio del sistema, permitiendo que el controlador únicamente delegue la petición. El servicio se encarga de crear la entidad, establecer sus valores iniciales y coordinar la persistencia a través del repositorio, garantizando una adecuada separación de responsabilidades y bajo acoplamiento entre capas.

Justificación

La aplicación pasa de un diseño monolítico acoplado a una arquitectura estructurada en capas:

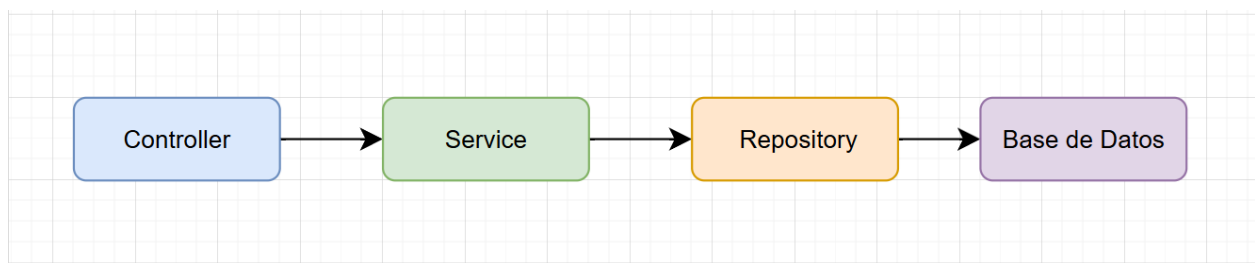


Ilustración 4-Diagrama de arquitectura por capas del sistema (Controller-Service-Repository-Base de Datos).

Esto permite:

- Separación clara de responsabilidades.
- Mayor mantenibilidad.
- Mejor testabilidad.
- Preparación para microservicios.

3.2 Repository Pattern (Eliminar SQL Directo)

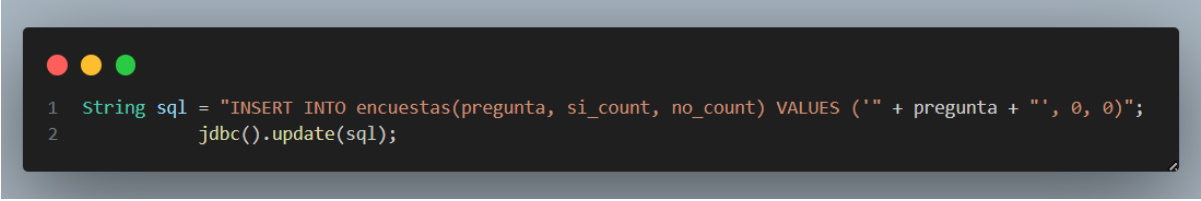
En la implementación actual, el controlador EncuestaController no solo gestiona las peticiones HTTP, sino que también construye manualmente consultas SQL y crea instancias de JdbcTemplate para ejecutar operaciones contra la base de datos.

Este diseño implica que la capa web conoce detalles de infraestructura y persistencia, generando un fuerte acoplamiento entre la lógica de negocio y el motor de base de datos.

A screenshot of a code editor with a dark background and light-colored text. The code is in Java and shows a method named jdbc() that creates a DriverManagerDataSource and returns a new JdbcTemplate instance. The code is numbered from 1 to 10 on the left side. The code is as follows:

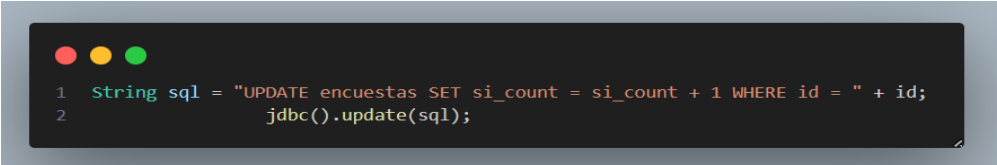
```
1  CÓDIGO ACTUAL (ANTI-PATRÓN )
2  private JdbcTemplate jdbc() {
3      DriverManagerDataSource ds = new DriverManagerDataSource();
4      ds.setDriverClassName("org.postgresql.Driver");
5      ds.setUrl(URL);
6      ds.setUsername(USER);
7      ds.setPassword(PASS);
8      return new JdbcTemplate(ds);
9  }
10
```

Ilustración 5-Creación manual del DataSource



```
1 String sql = "INSERT INTO encuestas(pregunta, si_count, no_count) VALUES ('" + pregunta + "', 0, 0)";
2 jdbc().update(sql);
```

Ilustración 6-SQL concatenado en método crear()



```
1 String sql = "UPDATE encuestas SET si_count = si_count + 1 WHERE id = " + id;
2 jdbc().update(sql);
```

Ilustración 7-SQL dinámico en votar()

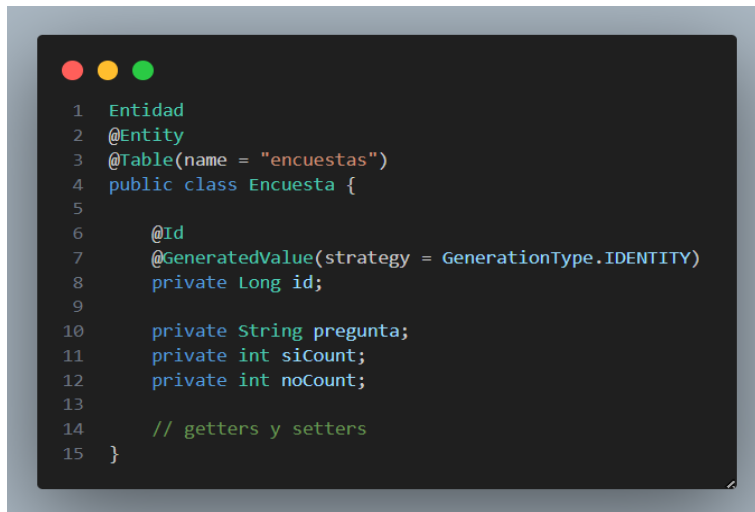
Problemas Detectados

1. SQL construido mediante concatenación de strings.
2. Vulnerabilidad potencial a SQL Injection.
3. Acoplamiento entre capa web y base de datos.
4. Repetición de lógica SQL en múltiples métodos.
5. Instanciación manual del DataSource en cada llamada.
6. No existe capa de persistencia separada.
7. Difícil mantenimiento y escalabilidad.

Arquitectónicamente, el controlador actúa como Controller + Repository al mismo tiempo

Propuesta con Repository Pattern + Spring Data JPA

Se propone separar la persistencia en una capa dedicada utilizando Spring Data JPA, eliminando la construcción manual de consultas SQL y el uso directo de JdbcTemplate dentro del controlador.

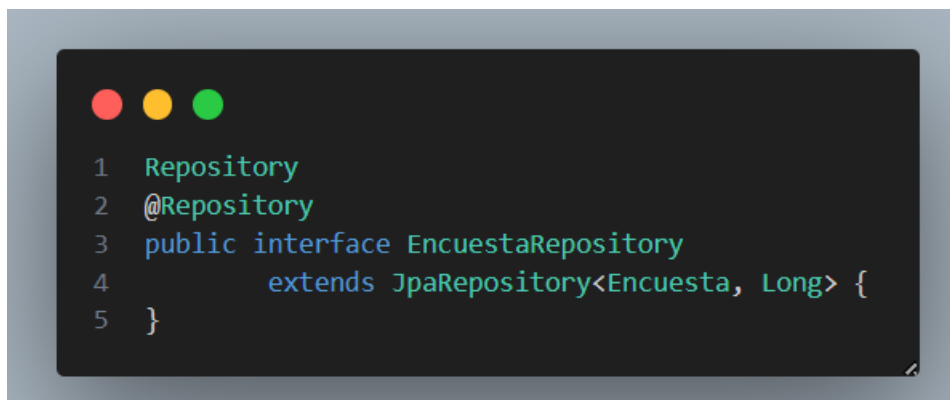


```
1  Entidad
2  @Entity
3  @Table(name = "encuestas")
4  public class Encuesta {
5
6      @Id
7      @GeneratedValue(strategy = GenerationType.IDENTITY)
8      private Long id;
9
10     private String pregunta;
11     private int siCount;
12     private int noCount;
13
14     // getters y setters
15 }
```

Ilustración 8- Implementación de la entidad utilizando Repository Pattern con Spring Data JPA

Esta clase representa la tabla encuestas en la base de datos.

Ahora la tabla deja de manipularse mediante SQL manual y pasa a mapearse como objeto Java.



```
1  Repository
2  @Repository
3  public interface EncuestaRepository
4      extends JpaRepository<Encuesta, Long> {
5  }
```

Ilustración 9-Implementación del Repository Pattern mediante Spring Data JPA.

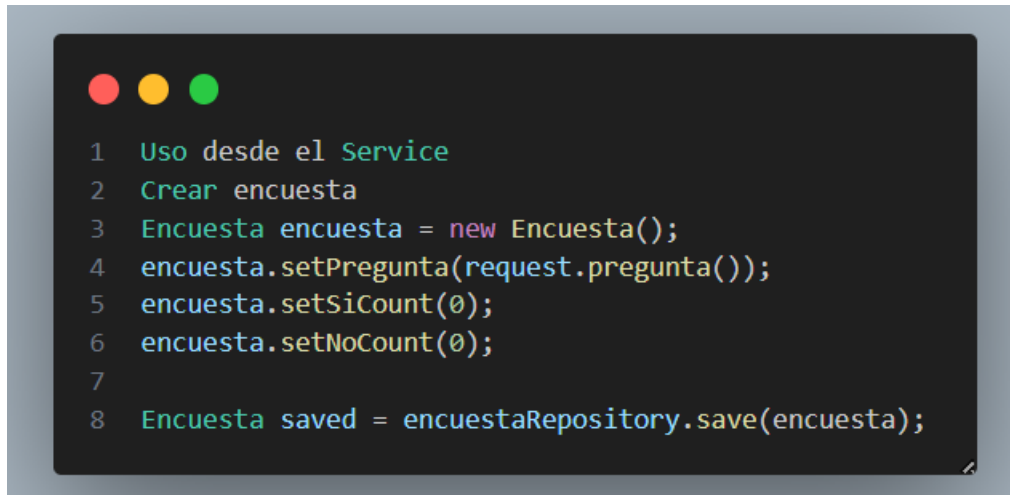
Esta interfaz reemplaza completamente el uso de:

- JdbcTemplate
- DriverManagerDataSource
- Construcción manual de consultas SQL

Spring Data JPA se encarga automáticamente de:

- Generar consultas

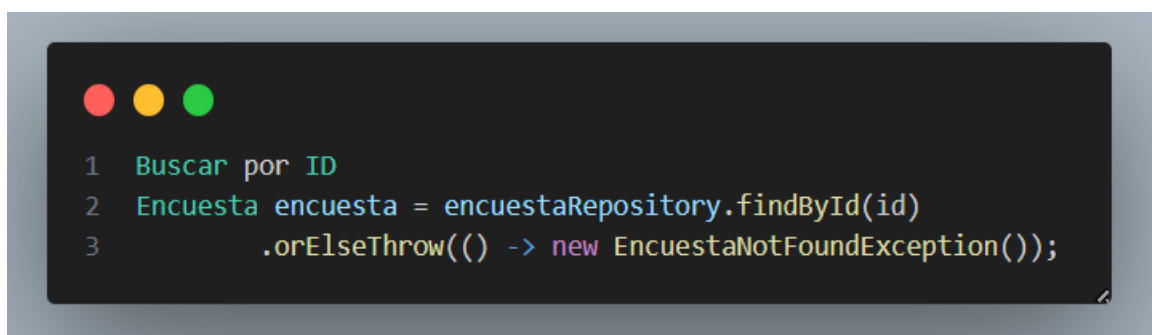
- Manejar conexiones
- Usar prepared statements
- Mapear resultados a objetos



```
1  Uso desde el Service
2  Crear encuesta
3  Encuesta encuesta = new Encuesta();
4  encuesta.setPregunta(request.pregunta());
5  encuesta.setSiCount(0);
6  encuesta.setNoCount(0);
7
8  Encuesta saved = encuestaRepository.save(encuesta);
```

Ilustración 10- Uso del repositorio desde la capa Service para la persistencia de la entidad.

La figura muestra cómo la capa Service crea la entidad y delega su almacenamiento al repositorio mediante el método `save()`. Este proceso permite mantener la lógica de negocio separada de la capa de persistencia, cumpliendo con el principio de responsabilidad única y promoviendo una arquitectura desacoplada.

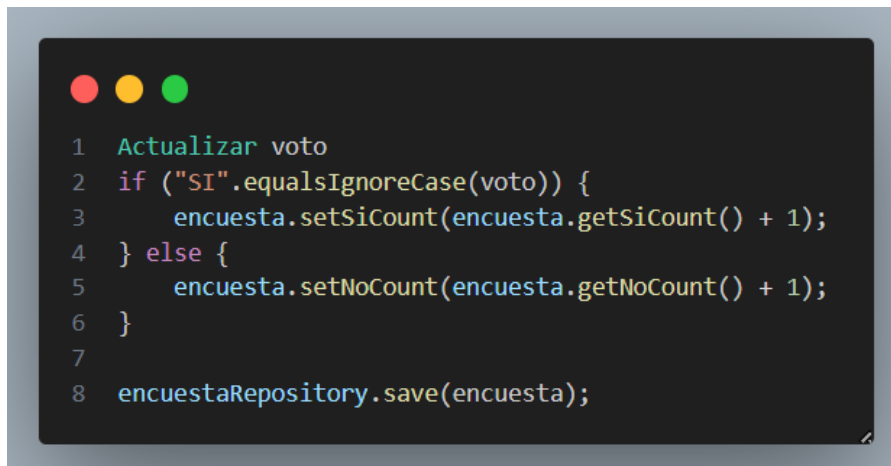


```
1  Buscar por ID
2  Encuesta encuesta = encuestaRepository.findById(id)
3      .orElseThrow(() -> new EncuestaNotFoundException());
```

Ilustración 11- Consulta de entidad por ID con manejo de excepción en la capa Service.

La figura presenta el uso del método `findById()` proporcionado por Spring Data JPA para consultar una entidad por su identificador. En caso de no encontrarse el registro, se lanza una excepción

personalizada, lo que permite un manejo adecuado de errores dentro de la capa Service y mantiene la lógica de control centralizada.

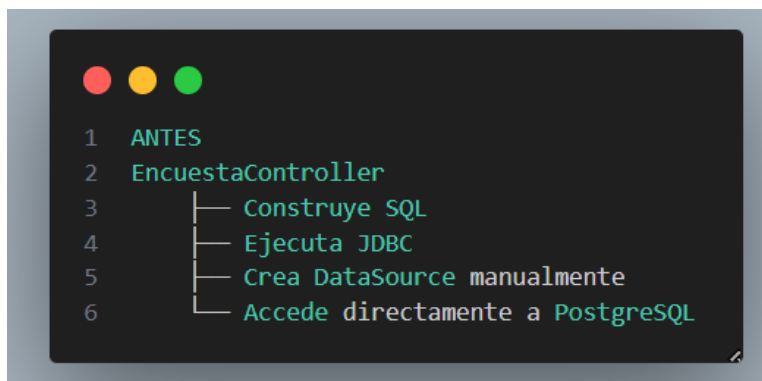


```
1  Actualizar voto
2  if ("SI".equalsIgnoreCase(voto)) {
3      encuesta.setSiCount(encuesta.getSiCount() + 1);
4  } else {
5      encuesta.setNoCount(encuesta.getNoCount() + 1);
6  }
7
8  encuestaRepository.save(encuesta);
```

Ilustración 12-Actualización de voto y persistencia de cambios en la capa Service.

La figura muestra la lógica de actualización del conteo de votos dentro de la capa Service, donde se modifica el estado de la entidad y posteriormente se persiste el cambio mediante el repositorio.

Cambio Estructural



```
1  ANTES
2  EncuestaController
3      |— Construye SQL
4      |— Ejecuta JDBC
5      |— Crea DataSource manualmente
6      |— Accede directamente a PostgreSQL
```

Ilustración 13-Estructura inicial del sistema antes de aplicar arquitectura por capas.

El controlador tiene conocimiento directo de la base de datos.



Ilustración 14- Estructura del sistema después de aplicar arquitectura por capas.

Ahora:

- El Controller solo gestiona HTTP.
- El Service contiene la lógica de negocio.
- El Repository gestiona persistencia.
- La base de datos queda desacoplada.

Justificación Técnica

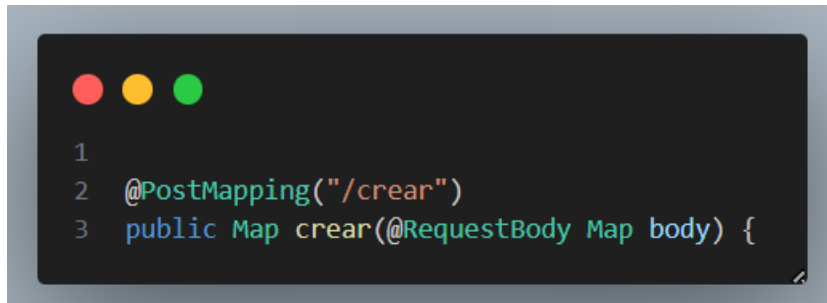
- La aplicación del Repository Pattern aporta:
- Eliminación de SQL Injection.
- Desacoplamiento de la capa web respecto a la base de datos.
- Eliminación de creación manual de conexiones.
- Centralización del acceso a datos.
- Mayor mantenibilidad.
- Mejor testabilidad.
- Preparación para microservicios (cada servicio tendrá su propio repositorio).
- Reducción de código repetitivo.

- Uso automático de prepared statements mediante JPA.

3.3 DTO Pattern (Data Transfer Object)

En la implementación actual del sistema, los endpoints del controlador EncuestaController reciben y retornan información utilizando estructuras genéricas de tipo Map.

Ejemplo real del código actual:

A screenshot of a code editor with a dark background and light-colored text. The code is a Java endpoint definition for a POST request. It starts with a line number '1' followed by a blank line. Line '2' contains the annotation '@PostMapping("/crear")'. Line '3' contains the method signature 'public Map crear(@RequestBody Map body) {' followed by an opening curly brace. The code is color-coded: '@' is blue, 'PostMapping' is green, the path is in quotes, 'public' is blue, 'Map' is green, 'crear' is blue, '@RequestBody' is blue, 'Map' is green, and 'body' is blue. There are three colored circles (red, yellow, green) in the top left corner of the editor window.

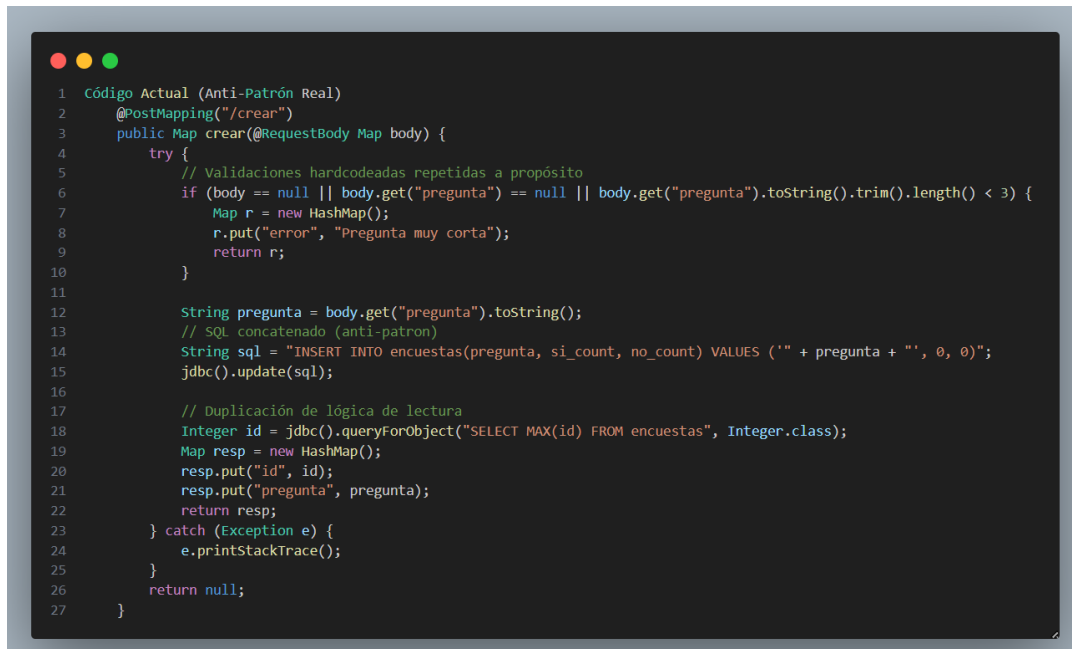
```
1
2 @PostMapping("/crear")
3 public Map crear(@RequestBody Map body) {
```

Ilustración 15- Definición del endpoint POST para la creación de encuestas.

Este diseño presenta múltiples problemas arquitectónicos y de calidad de código:

- No existe un contrato formal entre frontend y backend.
- No hay validación automática de datos.
- Se pierde el tipado fuerte de Java.
- El compilador no puede detectar errores estructurales.
- El IDE no puede asistir con autocompletado semántico.
- Se incrementa el riesgo de errores en tiempo de ejecución.

Desde una perspectiva de diseño, esto constituye un anti-patrón de tipado débil, contrario a los principios de robustez y claridad estructural.



```
1  Código Actual (Anti-Patrón Real)
2  @PostMapping("/crear")
3  public Map crear(@RequestBody Map body) {
4      try {
5          // Validaciones hardcodeadas repetidas a propósito
6          if (body == null || body.get("pregunta") == null || body.get("pregunta").toString().trim().length() < 3) {
7              Map r = new HashMap();
8              r.put("error", "Pregunta muy corta");
9              return r;
10         }
11
12         String pregunta = body.get("pregunta").toString();
13         // SQL concatenado (anti-patron)
14         String sql = "INSERT INTO encuestas(pregunta, si_count, no_count) VALUES ('" + pregunta + "', 0, 0)";
15         jdbc().update(sql);
16
17         // Duplicación de lógica de lectura
18         Integer id = jdbc().queryForObject("SELECT MAX(id) FROM encuestas", Integer.class);
19         Map resp = new HashMap();
20         resp.put("id", id);
21         resp.put("pregunta", pregunta);
22         return resp;
23     } catch (Exception e) {
24         e.printStackTrace();
25     }
26     return null;
27 }
```

Ilustración 16- Código actual con anti-patronos y alta concentración de responsabilidades en el controlador.

Problemas Detectados

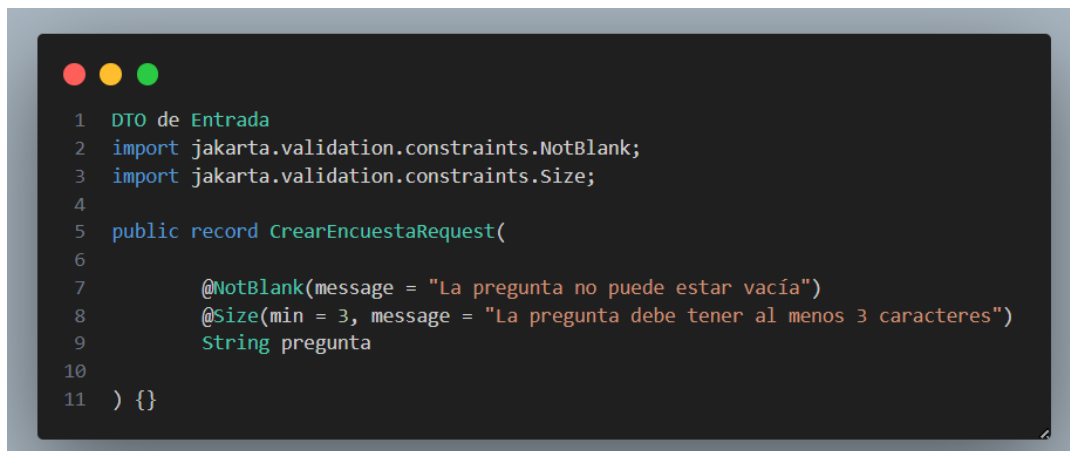
- Uso de Map en lugar de clases tipadas.
- Validaciones manuales repetitivas.
- Construcción manual de respuesta.
- Ausencia de contrato explícito.
- Sin validación declarativa.
- Difícil documentación de API.
- No preparado para OpenAPI / Swagger.

Propuesta de Implementación con DTO Pattern

El patrón DTO (Data Transfer Object) define clases específicas para representar los datos que se transfieren entre capas.

En lugar de usar Map, se crean objetos tipados:

- DTO de entrada (Request)
- DTO de salida (Response)

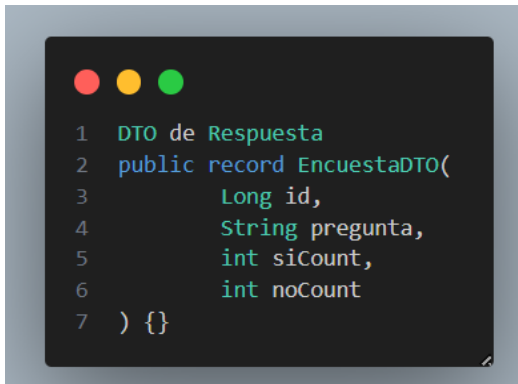


```
1 DTO de Entrada
2 import jakarta.validation.constraints.NotBlank;
3 import jakarta.validation.constraints.Size;
4
5 public record CrearEncuestaRequest(
6
7     @NotBlank(message = "La pregunta no puede estar vacía")
8     @Size(min = 3, message = "La pregunta debe tener al menos 3 caracteres")
9     String pregunta
10
11 ) {}
```

Ilustración 17- Implementación del DTO de entrada con validaciones mediante Jakarta Validation.

¿Qué cambia aquí?

- Se define explícitamente qué campos recibe el endpoint.
- Se habilita validación automática.
- Se elimina validación manual dentro del controlador.
- Se establece contrato formal.



```
1 DTO de Respuesta
2 public record EncuestaDTO(
3     Long id,
4     String pregunta,
5     int siCount,
6     int noCount
7 ) {}
```

Ilustración 18- Implementación del DTO de respuesta para la transferencia de datos al cliente.

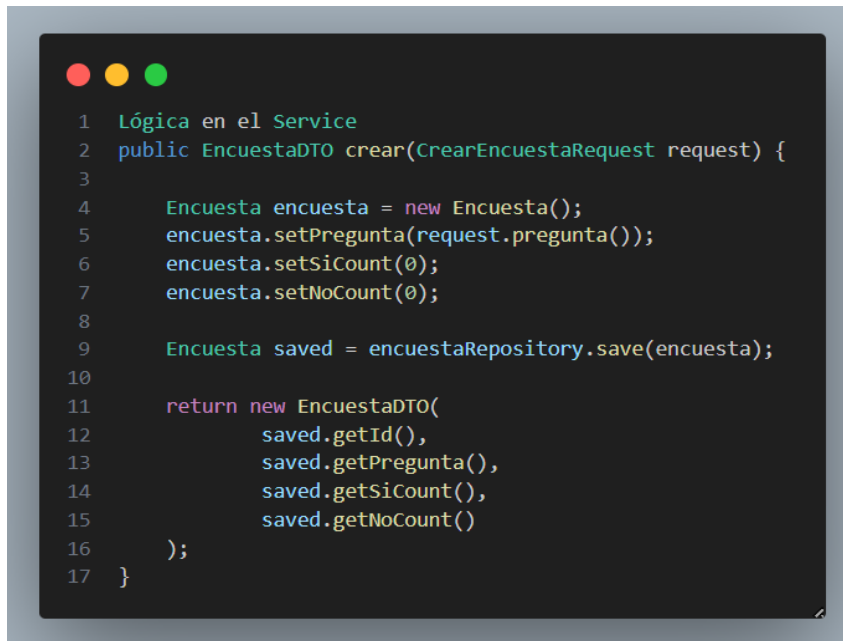
Este DTO define claramente la estructura de la respuesta que recibe el frontend.



```
1 Nuevo Controller Refactorizado
2 @PostMapping("/crear")
3 public ResponseEntity<EncuestaDTO> crear(
4     @Valid @RequestBody CrearEncuestaRequest request) {
5
6     EncuestaDTO response = encuestaService.crear(request);
7
8     return ResponseEntity.ok(response);
9 }
```

Ilustración 19- Controlador refactorizado aplicando arquitectura por capas y validación de entrada.

La figura presenta la versión refactorizada del controlador, donde se aplica una correcta separación de responsabilidades. En esta implementación, el Controller se limita a recibir la solicitud HTTP, validar el DTO de entrada mediante `@Valid` y delegar la lógica de negocio a la capa Service. De esta manera, se elimina el acceso directo a la base de datos y se promueve una arquitectura más limpia, mantenible y desacoplada.

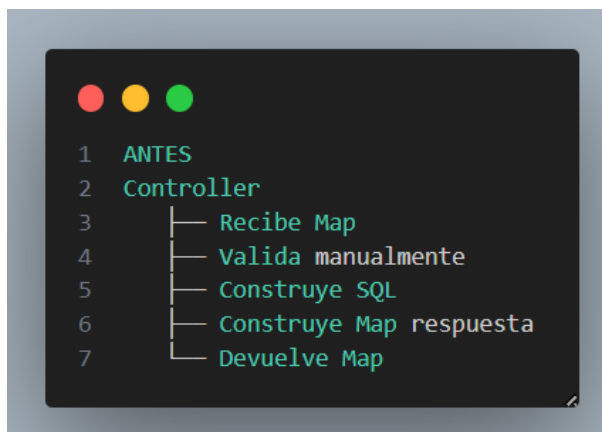


```
1  Lógica en el Service
2  public EncuestaDTO crear(CrearEncuestaRequest request) {
3
4      Encuesta encuesta = new Encuesta();
5      encuesta.setPregunta(request.pregunta());
6      encuesta.setSiCount(0);
7      encuesta.setNoCount(0);
8
9      Encuesta saved = encuestaRepository.save(encuesta);
10
11     return new EncuestaDTO(
12         saved.getId(),
13         saved.getPregunta(),
14         saved.getSiCount(),
15         saved.getNoCount()
16     );
17 }
```

Ilustración 20- Implementación de la lógica de negocio en la capa Service.

En esta sección se evidencia la implementación de la lógica de negocio dentro de la capa Service. El método se encarga de crear la entidad, asignar sus valores iniciales y delegar la operación de persistencia al repositorio mediante Spring Data JPA. Finalmente, se transforma la entidad guardada en un DTO de respuesta, garantizando una correcta separación entre lógica de negocio, acceso a datos y presentación.

Cambio Estructural



```
1  ANTES
2  Controller
3      |— Recibe Map
4      |— Valida manualmente
5      |— Construye SQL
6      |— Construye Map respuesta
7      |— Devuelve Map
```

Ilustración 21- Flujo de responsabilidades del Controller antes de la refactorización.

La figura representa la versión inicial del sistema, donde el Controller asume múltiples responsabilidades, incluyendo validación, construcción de SQL y generación de respuesta, lo que genera alto acoplamiento y baja mantenibilidad.

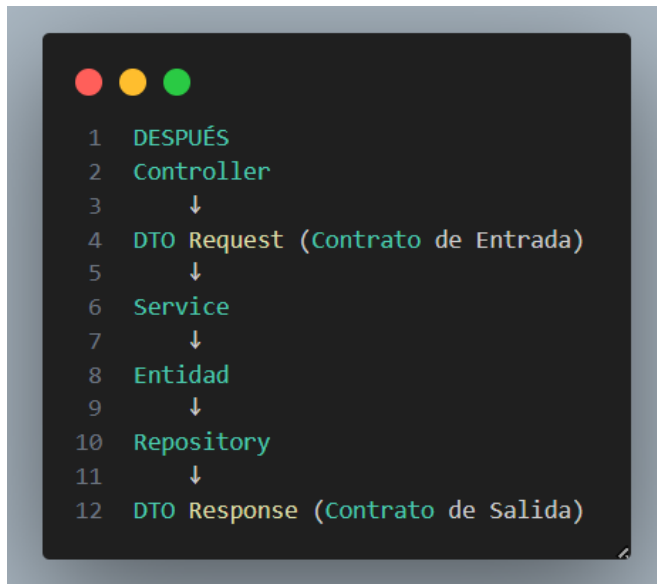


Ilustración 22- Flujo de responsabilidades después de la refactorización.

La figura presenta el flujo de responsabilidades después de la refactorización, evidenciando una clara separación por capas. El Controller recibe la petición y utiliza un DTO como contrato de entrada, delegando la lógica de negocio al Service. Posteriormente, la entidad es gestionada por el Repository para su persistencia y finalmente se construye un DTO de respuesta como contrato de salida. Esta organización reduce el acoplamiento y mejora la mantenibilidad, escalabilidad y claridad estructural del sistema.

Beneficios Arquitectónicos

La implementación del DTO Pattern aporta:

- Contrato formal entre capas.
- Validación automática con anotaciones.

- Tipado fuerte en tiempo de compilación.
- Eliminación de validaciones repetidas.
- Mejor integración con Swagger.
- Mayor claridad semántica.
- Código más legible.
- Preparación para microservicios.
- Separación clara entre modelo de dominio y modelo de transferencia.

Impacto en Clean Code

El uso de DTOs contribuye directamente a:

- Nombres significativos.
- Eliminación de lógica repetitiva.
- Código autoexplicativo.
- Reducción de complejidad en el controlador.

Conexión con Microservicios

En una arquitectura de microservicios, los DTOs se convierten en contratos explícitos entre servicios. Definirlos correctamente desde el monolito facilita:

- Versionado de API.
- Evolución independiente de servicios.
- Comunicación segura entre bounded contexts.

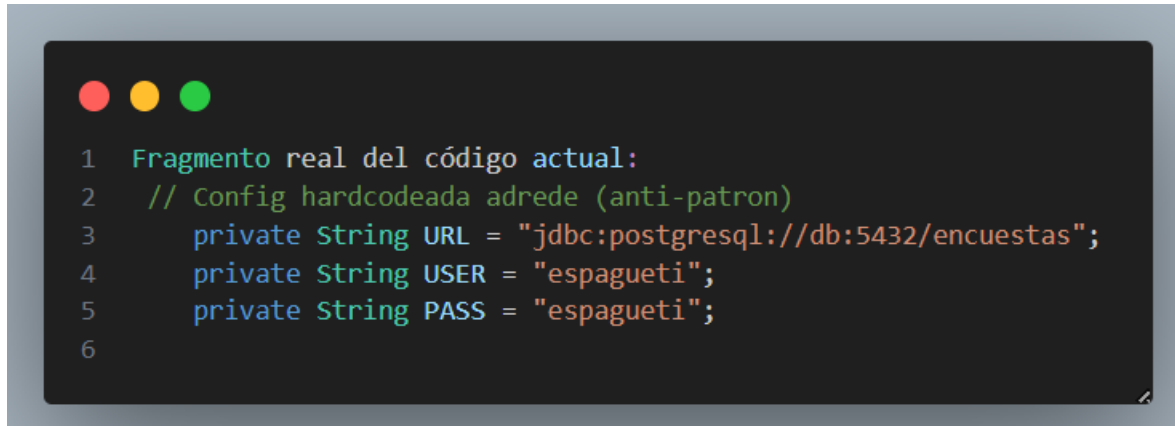
Conclusión del DTO Pattern

La migración de Map a objetos DTO tipados representa una mejora estructural significativa. Se pasa de un modelo dinámico y débilmente tipado a uno robusto, validado y profesional.

Este cambio no solo mejora la calidad del código actual, sino que establece las bases necesarias para una arquitectura escalable y mantenible.

3.4 Externalized Configuration (Eliminación de Configuración Hardcodeada)

En la implementación actual del sistema, las credenciales y la configuración de conexión a la base de datos se encuentran definidas directamente dentro del código fuente del controlador EncuestaController.



```
1  Fragmento real del código actual:
2  // Config hardcodeada adrede (anti-patron)
3      private String URL = "jdbc:postgresql://db:5432/encuestas";
4      private String USER = "espagueti";
5      private String PASS = "espagueti";
6
```

Ilustración 23- Fragmento del código actual con configuración hardcodeada (anti-patrón).

La clase conoce directamente:

- Motor de base de datos
- Host
- Puerto
- Nombre de base
- Usuario
- Contraseña

Esto viola el principio de inversión de dependencias (DIP).

Además, el método jdbc() crea manualmente el DriverManagerDataSource utilizando estas variables:

A screenshot of a code editor with a dark background and light-colored text. The code is a Java method named `jdbc()` inside a class. It creates a `DriverManagerDataSource` object, sets its properties (driver class name, URL, username, password), and returns a new `JdbcTemplate` instance. The code is as follows:

```
1 private JdbcTemplate jdbc() {  
2     DriverManagerDataSource ds = new DriverManagerDataSource();  
3     ds.setDriverClassName("org.postgresql.Driver");  
4     ds.setUrl(URL);  
5     ds.setUsername(USER);  
6     ds.setPassword(PASS);  
7     return new JdbcTemplate(ds);  
8 }
```

Ilustración 24- Creación manual del DriverManagerDataSource en el controlador (anti-patrón).

Problemas Detectados

La configuración hardcodeada presenta múltiples riesgos:

- Exposición de credenciales sensibles en el código.
- Imposibilidad de cambiar ambiente (dev, qa, prod) sin modificar el código.
- Violación del principio de separación entre configuración e implementación.
- Dificultad para despliegue en contenedores Docker.
- Riesgo de filtración en repositorios públicos.
- Alto acoplamiento entre infraestructura y lógica de aplicación.

Desde el punto de vista arquitectónico, este diseño impide la escalabilidad y dificulta la transición hacia microservicios.

Propuesta de Solución

Se propone aplicar el patrón Externalized Configuration, el cual consiste en separar completamente la configuración del código fuente.

En Spring Boot, esto se logra mediante:

- `application.yml`
- Variables de entorno
- Configuración automática del DataSource

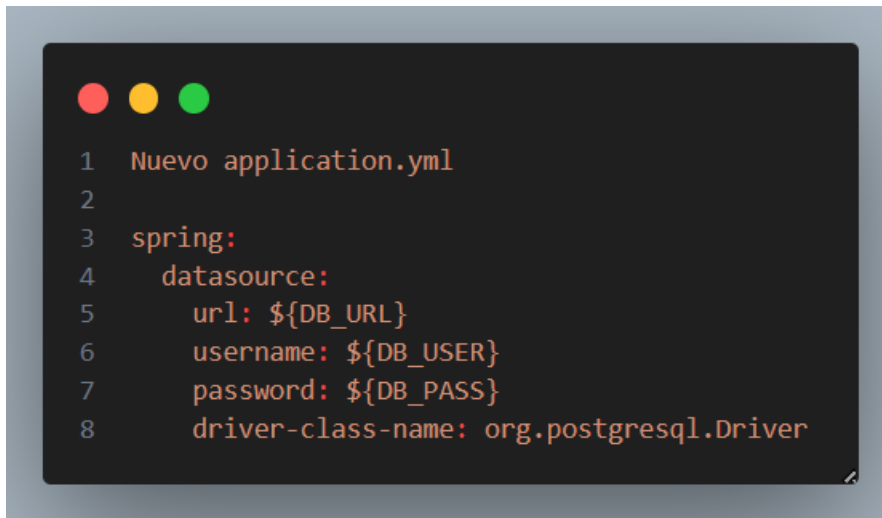


Ilustración 25- Configuración externa de base de datos mediante application.yml.

La figura muestra la externalización de la configuración de la base de datos mediante el archivo application.yml, utilizando variables de entorno para las credenciales. Esta práctica elimina el hardcodeo en el código fuente, mejora la seguridad y permite una gestión adecuada de entornos (desarrollo, pruebas y producción).



Ilustración 26- Definición de variables de entorno en Docker para la configuración de la base de datos.

La figura muestra la definición de variables de entorno dentro del contenedor Docker, las cuales son consumidas por el archivo application.yml. Esta integración permite separar completamente la configuración del código fuente, facilitando el despliegue en distintos entornos y mejorando la seguridad y portabilidad de la aplicación.

Eliminación del método jdbc()

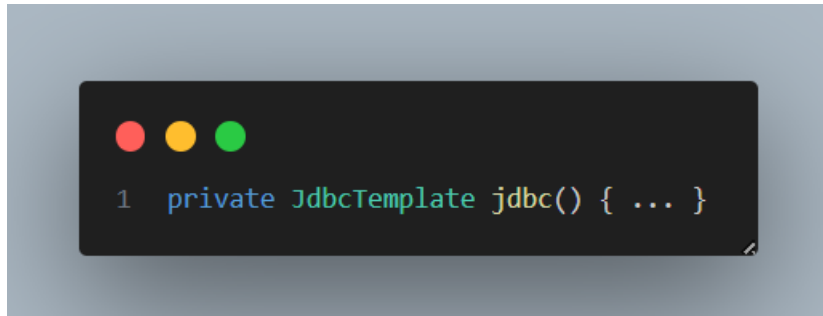


Ilustración 27- Método jdbc() eliminado tras adoptar la autoconfiguración de Spring Boot.

La figura representa el método jdbc() que es eliminado en la versión refactorizada. Con la adopción de Spring Boot y Spring Data JPA, el DataSource es autoconfigurado automáticamente, y el acceso a datos se delega al EncuestaRepository, eliminando la necesidad de crear manualmente conexiones JDBC.

Cambio Estructural

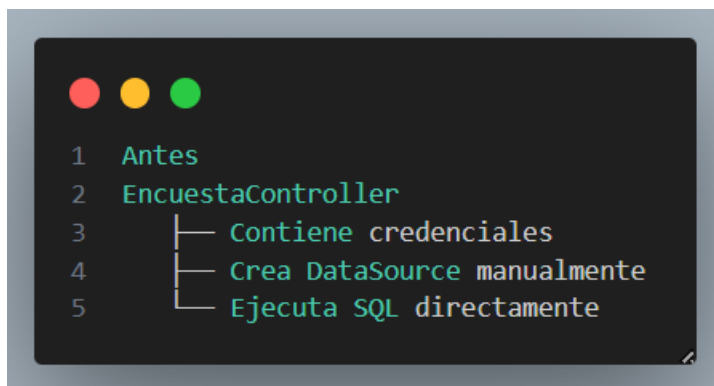


Ilustración 28- Responsabilidades concentradas en el Controller antes de la refactorización.

La figura resume la estructura inicial del sistema, donde el controlador contiene credenciales, crea manualmente el DataSource y ejecuta sentencias SQL directamente. Esta implementación evidencia un alto acoplamiento y una clara ausencia de separación de responsabilidades.

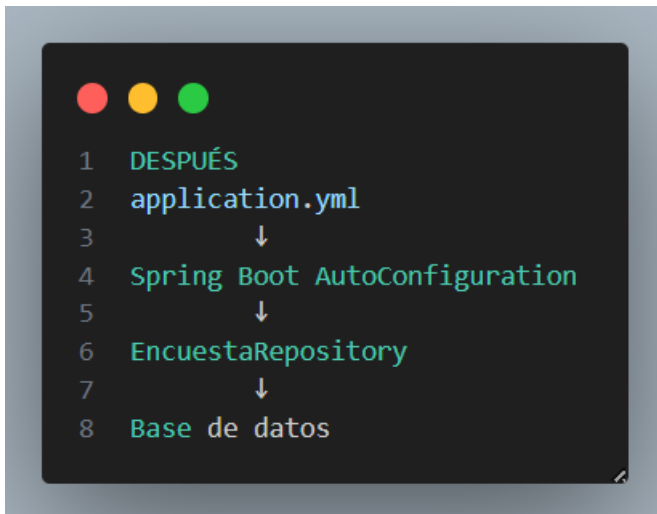


Ilustración 29- Flujo de configuración y acceso a datos después de la refactorización.

La figura muestra la estructura posterior a la refactorización, donde la configuración de base de datos se externaliza en `application.yml`, Spring Boot autoconfigura el `DataSource` y el acceso a datos se realiza exclusivamente a través del `EncuestaRepository`, eliminando la gestión manual de conexiones y reduciendo el acoplamiento.

Beneficios Arquitectónicos

La aplicación del patrón Externalized Configuration permite:

- Separación entre código y configuración.
- Mayor seguridad.
- Soporte multi-ambiente.
- Compatibilidad con Docker y Kubernetes.
- Escalabilidad hacia microservicios.
- Cumplimiento de principios SOLID.
- Reducción del acoplamiento.

Impacto en Clean Code

Este cambio mejora:

- Separación de responsabilidades.
- Eliminación de código innecesario.
- Mayor claridad estructural.
- Reducción de dependencias técnicas en la capa web.

Conclusión del Patrón

La eliminación de configuración hardcodeada representa un paso crítico hacia una arquitectura profesional. Este cambio mejora significativamente la seguridad, mantenibilidad y portabilidad del sistema, alineándolo con buenas prácticas modernas de desarrollo backend.

3.5 Global Exception Handler (Manejo Centralizado de Errores)

En la implementación actual del sistema, el manejo de excepciones se realiza directamente dentro de cada método del controlador EncuestaController. Las excepciones son capturadas mediante bloques try-catch y se limitan a imprimir el error en consola utilizando `e.printStackTrace()`, retornando posteriormente null.

A screenshot of a code editor with a dark background and light-colored text. The code is a Java try-catch block. It starts with a comment in Spanish, followed by a catch block for Exception, which calls printStackTrace() and then returns null. The code is numbered 1 through 5.

```
1 Fragmento del código actual:
2 catch (Exception e) {
3     e.printStackTrace();
4 }
5 return null;
```

Ilustración 30- Manejo inadecuado de excepciones en la implementación inicial.

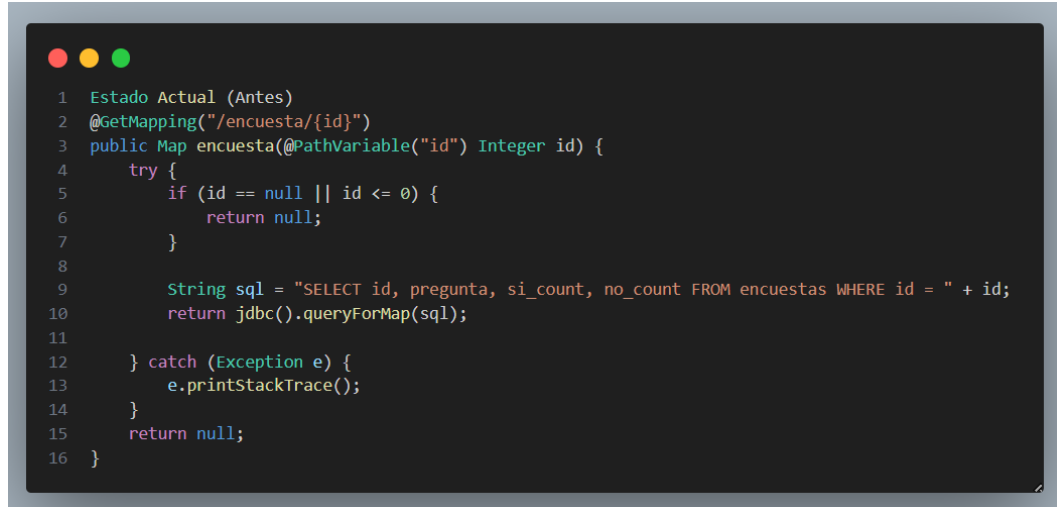
Este enfoque presenta múltiples problemas estructurales y arquitectónicos, ya que el controlador asume también la responsabilidad del manejo de errores, generando duplicación de lógica y respuestas HTTP inconsistentes.

Problemas Detectados

El manejo actual de errores presenta las siguientes deficiencias:

- Se retorna null ante excepciones.
- El cliente puede recibir un 200 OK incluso cuando ocurre un error.
- No existe una estructura estándar de respuesta de error.
- No se diferencian errores de negocio de errores del sistema.
- El código repite bloques try-catch en múltiples métodos.
- No existe trazabilidad clara de errores.

Desde el punto de vista arquitectónico, esto genera una API inconsistente y poco profesional.



```
1 Estado Actual (Antes)
2 @GetMapping("/encuesta/{id}")
3 public Map encuesta(@PathVariable("id") Integer id) {
4     try {
5         if (id == null || id <= 0) {
6             return null;
7         }
8
9         String sql = "SELECT id, pregunta, si_count, no_count FROM encuestas WHERE id = " + id;
10        return jdbc().queryForMap(sql);
11
12    } catch (Exception e) {
13        e.printStackTrace();
14    }
15    return null;
16 }
```

Ilustración 31- Implementación inicial del endpoint con manejo local de excepciones.

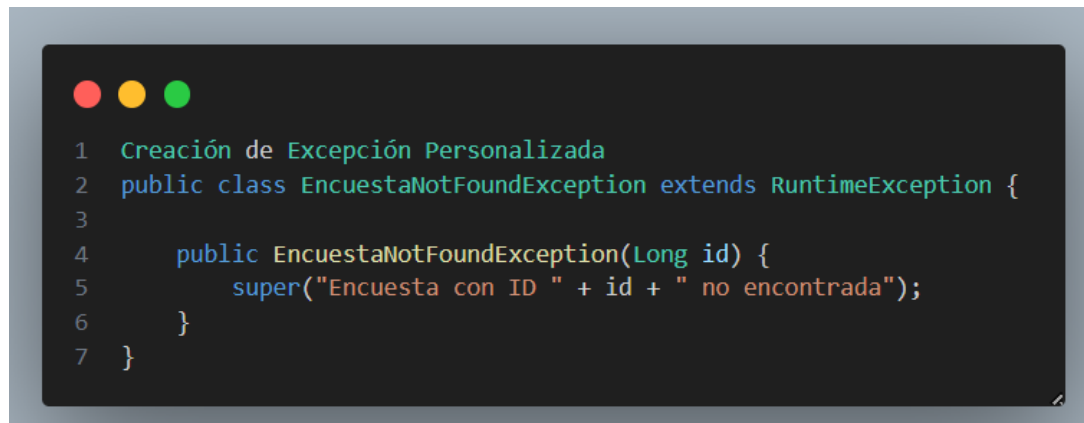
Problemas claros:

- Si ocurre un error en la consulta, se imprime en consola.
- Se devuelve null.

- No se retorna un código HTTP adecuado (404, 500, etc.).
- El frontend no puede diferenciar tipos de error.

Propuesta de Implementación

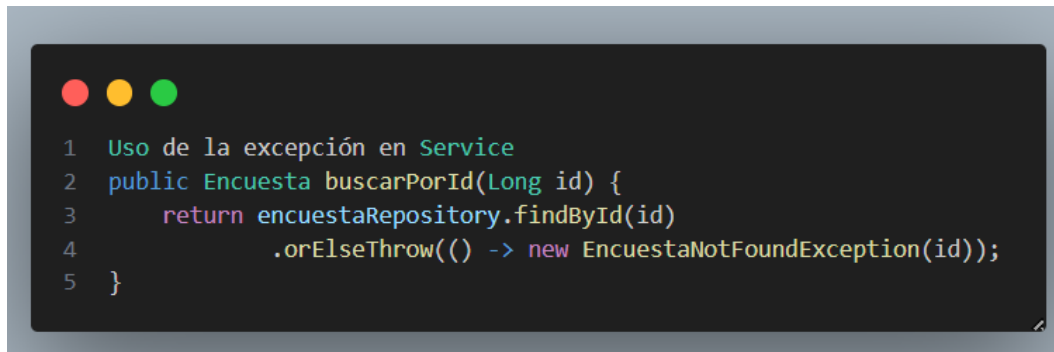
Se propone implementar el patrón Global Exception Handler mediante la anotación `@ControllerAdvice`, centralizando el manejo de errores en una única clase.



```
1  Creación de Excepción Personalizada
2  public class EncuestaNotFoundException extends RuntimeException {
3
4      public EncuestaNotFoundException(Long id) {
5          super("Encuesta con ID " + id + " no encontrada");
6      }
7  }
```

Ilustración 32- Creación de excepción personalizada para manejo controlado de errores.

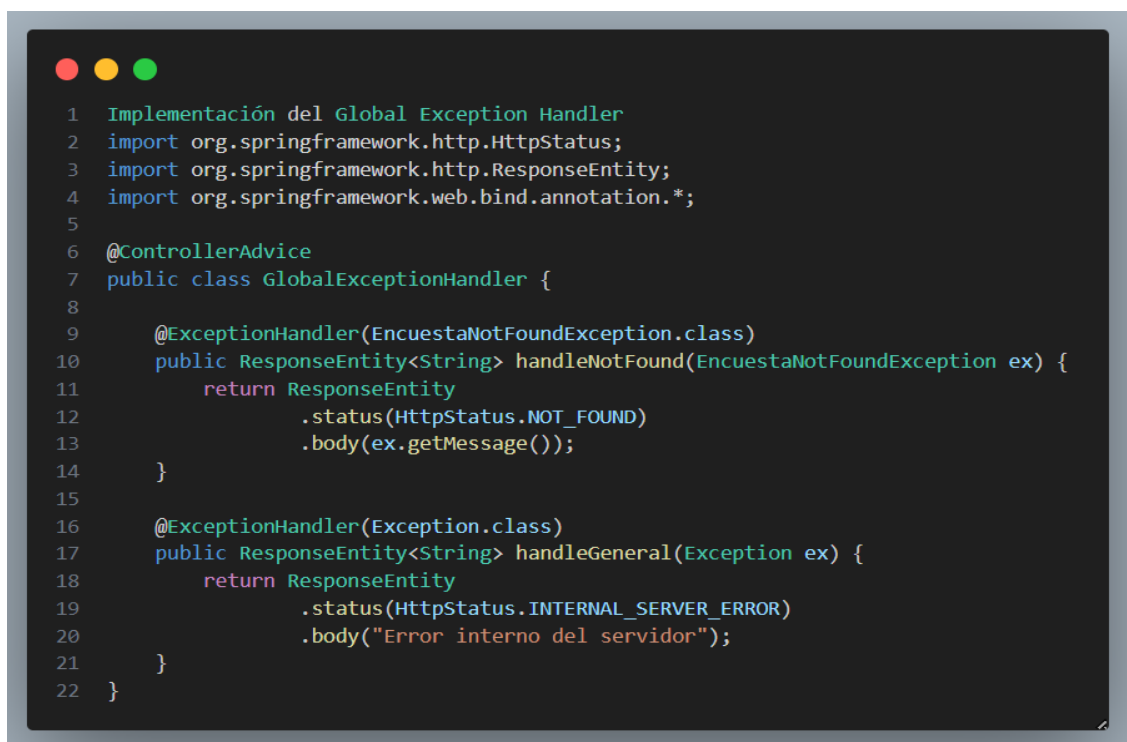
La figura muestra la creación de una excepción personalizada (`EncuestaNotFoundException`) que extiende `RuntimeException`. Esta implementación permite representar errores de negocio de manera explícita, evitando retornos nulos y facilitando su posterior manejo dentro de un Global Exception Handler. Con ello, se mejora la claridad semántica del código y se establece una base para un manejo centralizado y estructurado de errores.



```
1  Uso de la excepción en Service
2  public Encuesta buscarPorId(Long id) {
3      return encuestaRepository.findById(id)
4          .orElseThrow(() -> new EncuestaNotFoundException(id));
5  }
```

Ilustración 33- Uso de excepción personalizada en la capa Service mediante orElseThrow().

La figura muestra cómo la excepción personalizada es utilizada dentro de la capa Service mediante el método `orElseThrow()`. En caso de que la entidad no exista, se lanza una excepción específica, evitando retornos nulos y permitiendo que el error sea gestionado posteriormente por el Global Exception Handler. Esta práctica mejora la claridad del flujo lógico y fortalece el contrato de la API.



```
1  Implementación del Global Exception Handler
2  import org.springframework.http.HttpStatus;
3  import org.springframework.http.ResponseEntity;
4  import org.springframework.web.bind.annotation.*;
5
6  @ControllerAdvice
7  public class GlobalExceptionHandler {
8
9      @ExceptionHandler(EncuestaNotFoundException.class)
10     public ResponseEntity<String> handleNotFound(EncuestaNotFoundException ex) {
11         return ResponseEntity
12             .status(HttpStatus.NOT_FOUND)
13             .body(ex.getMessage());
14     }
15
16     @ExceptionHandler(Exception.class)
17     public ResponseEntity<String> handleGeneral(Exception ex) {
18         return ResponseEntity
19             .status(HttpStatus.INTERNAL_SERVER_ERROR)
20             .body("Error interno del servidor");
21     }
22 }
```

Ilustración 34- Implementación del Global Exception Handler mediante @ControllerAdvice.

La figura muestra la implementación del Global Exception Handler utilizando `@ControllerAdvice`, lo que permite centralizar el manejo de errores en la aplicación. Mediante métodos anotados con `@ExceptionHandler`, se capturan excepciones específicas y generales, retornando respuestas HTTP adecuadas y mensajes controlados. Esta estrategia elimina la repetición de bloques try-catch en los controladores, mejora la coherencia de las respuestas y fortalece la arquitectura del sistema.

Cambio Estructural

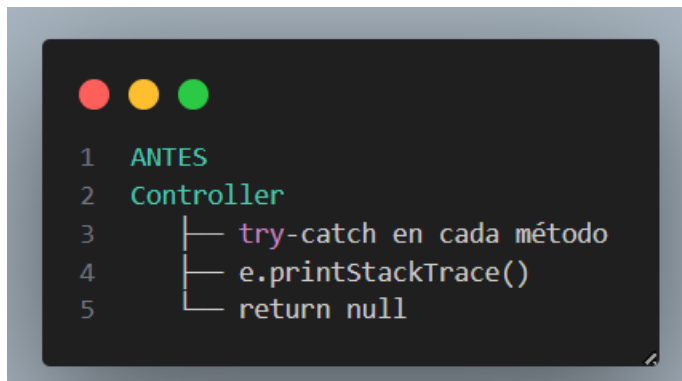


Ilustración 35- Manejo descentralizado de excepciones antes de la implementación global.

La figura muestra el manejo de excepciones en la versión inicial del sistema, donde cada método del controlador incluía bloques try-catch, utilizaba `printStackTrace()` y retornaba `null` ante errores. Esta estrategia generaba repetición de código, falta de estandarización en las respuestas y bajo control sobre los errores expuestos al cliente.

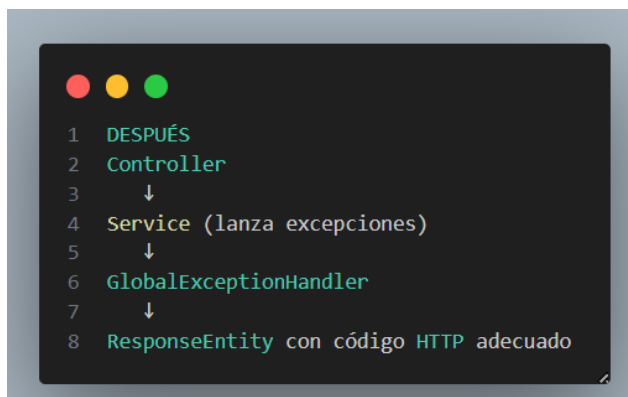


Ilustración 36-Flujo de manejo centralizado de excepciones después de la refactorización.

Ahora el manejo de errores:

- Está centralizado.
- Es consistente.
- Devuelve códigos HTTP correctos.
- No contamina la lógica del controlador.

Beneficios Arquitectónicos

La implementación de un Global Exception Handler aporta:

- Separación clara entre lógica de negocio y manejo de errores.
- Respuestas HTTP coherentes.
- Centralización del control de excepciones.
- Mejor experiencia para el cliente (frontend).
- API más profesional y estandarizada.
- Eliminación de duplicación de bloques try-catch.
- Preparación para microservicios.

Impacto en Clean Code

Este cambio contribuye a:

- Eliminación de código repetitivo.
- Mejora en legibilidad.
- Claridad estructural.
- Responsabilidad única en el controlador.
- Código más declarativo.

Conclusión del Patrón

La adopción del Global Exception Handler transforma el sistema de un modelo reactivo e inconsistente a una arquitectura profesional, robusta y preparada para producción.

Se pasa de un manejo de errores disperso y manual a un sistema centralizado, desacoplado y alineado con las mejores prácticas de Spring Boot.

3.6 Service Layer en Angular

Identificación del Anti-Patrón en el Proyecto

En el frontend actual, los componentes:

- crear.component.ts
- encuesta.component.ts
- respuestas.component.ts

inyectan directamente HttpClient y realizan las llamadas HTTP al backend.

Ejemplo real en crear.component.ts:

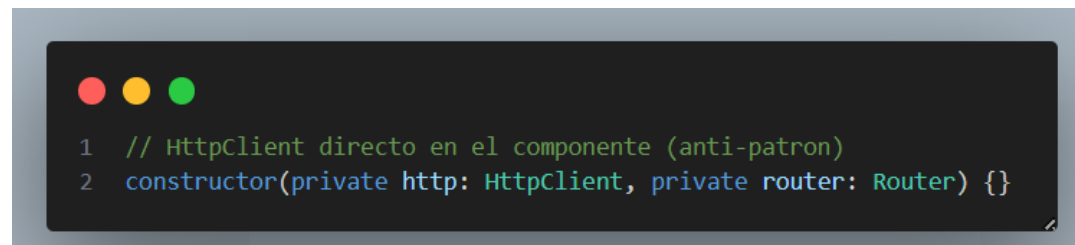
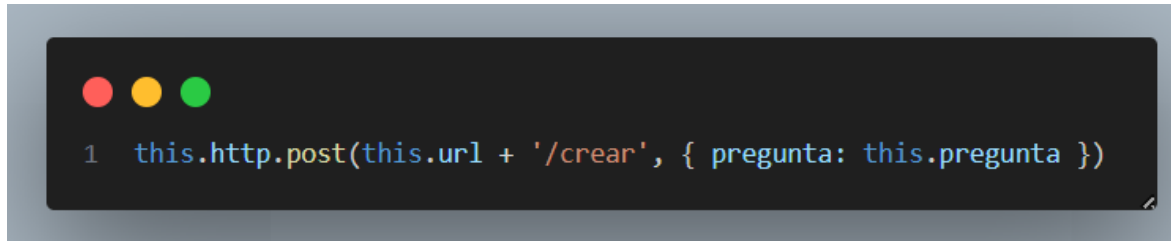


Ilustración 37-Uso directo de HttpClient en el componente (anti-patrón).

La figura muestra una implementación donde el componente Angular inyecta directamente HttpClient para realizar peticiones HTTP. Esta práctica genera acoplamiento entre la capa de presentación y la capa de acceso a datos, dificultando la reutilización, el mantenimiento y las pruebas unitarias del sistema.

Y más abajo:

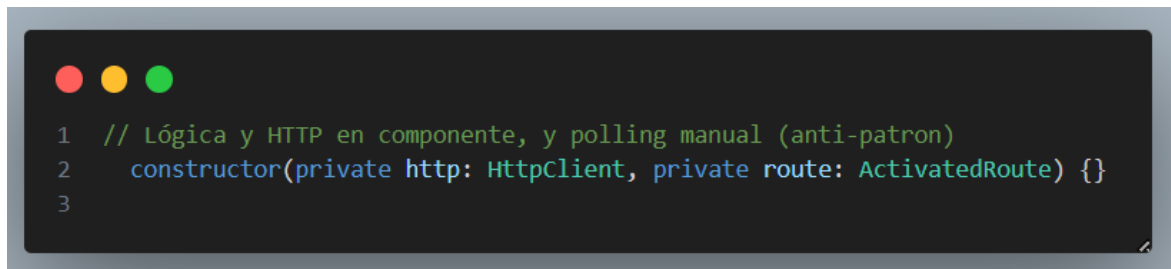


```
1 this.http.post(this.url + '/crear', { pregunta: this.pregunta })
```

Ilustración 38- Llamado HTTP directo desde el componente Angular (anti-patrón).

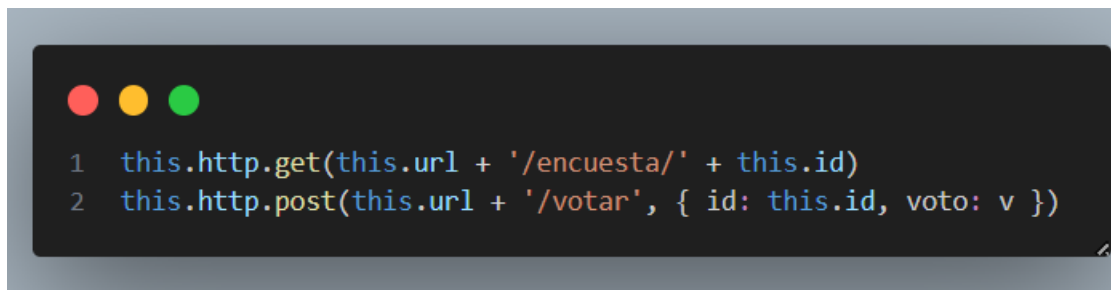
También se repite en:

[encuesta.component.ts](#)



```
1 // Lógica y HTTP en componente, y polling manual (anti-patron)
2 constructor(private http: HttpClient, private route: ActivatedRoute) {}
3
```

Ilustración 39- Uso de HttpClient y lógica de datos en encuesta.component.ts.



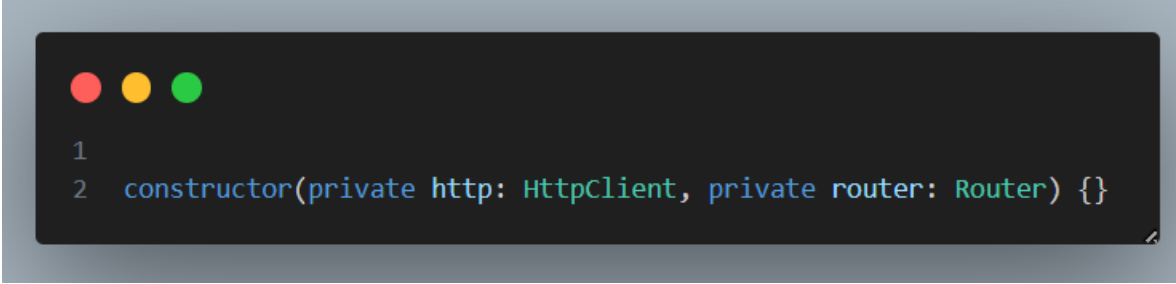
```
1 this.http.get(this.url + '/encuesta/' + this.id)
2 this.http.post(this.url + '/votar', { id: this.id, voto: v })
```

Ilustración 40-Consumo directo de endpoints /encuesta y /votar en el componente.

El componente Angular realiza directamente las llamadas HTTP para consultar y votar en una encuesta.

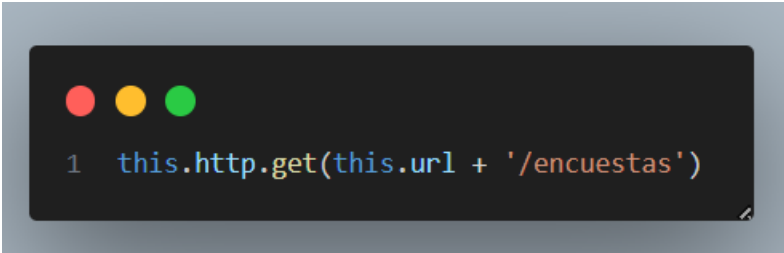
Esta implementación acopla la capa de presentación a la infraestructura de comunicación, dificultando la reutilización del código y centralizando múltiples responsabilidades en el componente. La ausencia de una capa de servicio intermedia afecta la mantenibilidad y la organización arquitectónica del frontend.

respuestas.component.ts



```
1  
2 constructor(private http: HttpClient, private router: Router) {}
```

Ilustración 41-Inyección directa de HttpClient en respuesta.component.ts (anti-patrón).

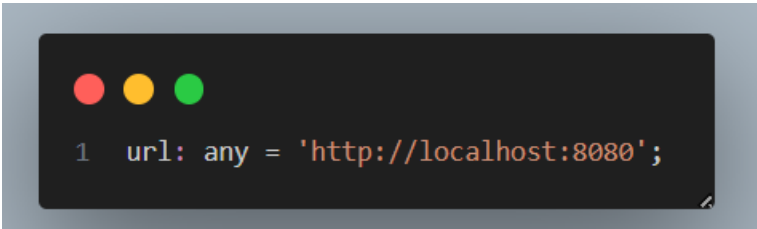


```
1 this.http.get(this.url + '/encuestas')
```

Ilustración 42- Consumo directo del endpoint /encuestas desde el componente Angular.

Problemas Arquitectónicos Detectados

1. URL hardcodeada en 3 componentes:



```
1 url: any = 'http://localhost:8080';
```

Ilustración 43-URL hardcodeada repetida en múltiples componentes Angular.

2. Lógica HTTP duplicada.
3. Acoplamiento fuerte entre vista y backend.
4. Violación del principio SRP.
5. Dificultad para migrar a microservicios.
6. Dificultad para agregar interceptores.
7. Uso excesivo de any.

8. Polling manual con setInterval en encuesta.component.

Estado Arquitectónico Actual

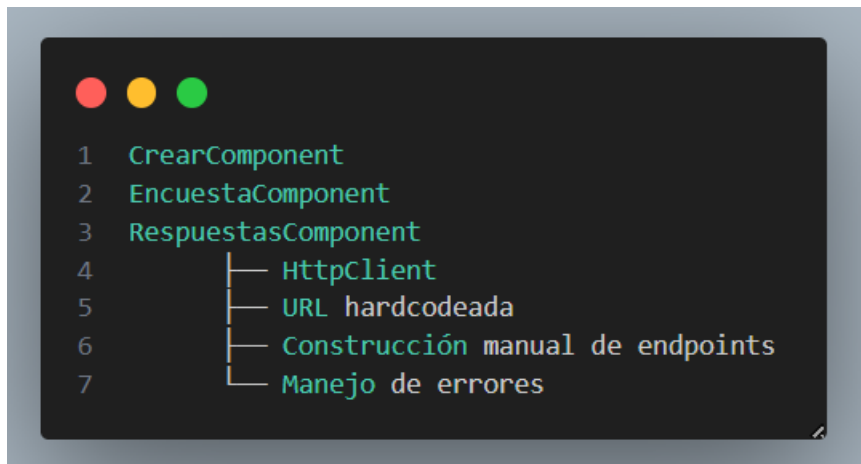


Ilustración 44- Componentes Angular con responsabilidades acopladas (anti-patrón).

El componente mezcla:

- Presentación
- Infraestructura
- Comunicación HTTP
- Lógica de negocio

Propuesta: Service Layer (Angular)

Se propone crear un servicio centralizado: encuesta.service.ts

Paso 1 – Centralizar URL en environment.ts

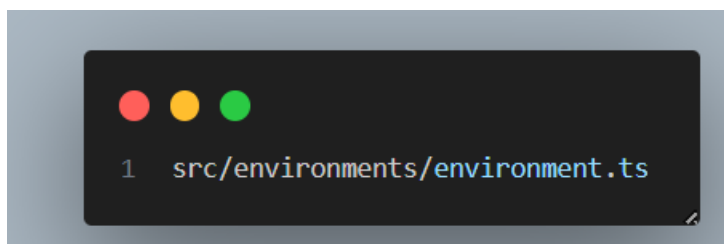
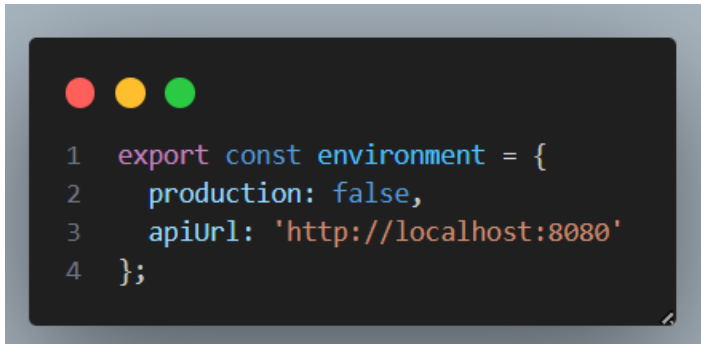


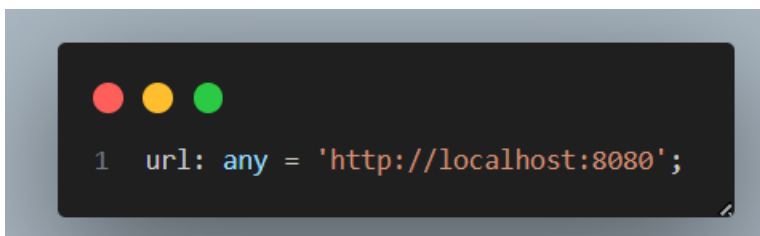
Ilustración 45-Centralización de la URL base en environment.ts (Paso 1 de la refactorización).



```
1 export const environment = {  
2   production: false,  
3   apiUrl: 'http://localhost:8080'  
4 };
```

Ilustración 46-Definición de la URL base en environment.ts.

Eliminar de los componentes:



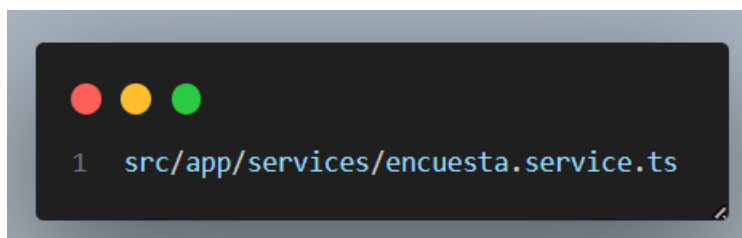
```
1 url: any = 'http://localhost:8080';
```

Ilustración 47-Eliminación de la URL hardcodeada en los componentes Angular.

La refactorización realizada permitió eliminar la configuración hardcodeada y la duplicación de la URL en múltiples componentes, centralizando la gestión del endpoint en environment.ts. Con ello, se redujo el acoplamiento al entorno local y se aplicó el principio DRY, mejorando la mantenibilidad del sistema.

Estos cambios preparan la aplicación para una arquitectura más limpia mediante la implementación de una Service Layer, promoviendo una mejor separación de responsabilidades y mayor escalabilidad del frontend.

Paso 2 – Crear EncuestaService



```
1 src/app/services/encuesta.service.ts
```

Ilustración 48-Creación del archivo encuesta.service.ts (Service Layer en Angular).

Como segundo paso de la refactorización, se crea el archivo encuesta.service.ts dentro del directorio src/app/services/. Este servicio tendrá la responsabilidad exclusiva de gestionar la comunicación HTTP con el backend, centralizando las llamadas a la API y eliminando la lógica de infraestructura de los componentes.

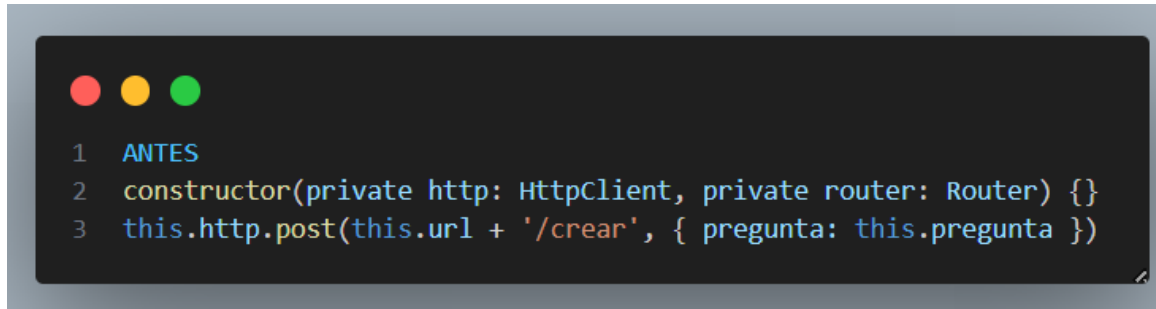


```
1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4 import { environment } from '../../environments/environment';
5
6 @Injectable({
7   providedIn: 'root'
8 })
9 export class EncuestaService {
10
11   private api = environment.apiUrl;
12
13   constructor(private http: HttpClient) {}
14
15   crear(pregunta: string): Observable<any> {
16     return this.http.post(`${this.api}/crear`, { pregunta });
17   }
18
19   obtenerTodas(): Observable<any> {
20     return this.http.get(`${this.api}/encuestas`);
21   }
22
23   obtenerPorId(id: number): Observable<any> {
24     return this.http.get(`${this.api}/encuesta/${id}`);
25   }
26
27   votar(id: number, voto: string): Observable<any> {
28     return this.http.post(`${this.api}/votar`, { id, voto });
29   }
30 }
```

Ilustración 49-Implementación de EncuestaService como capa de servicio en Angular.

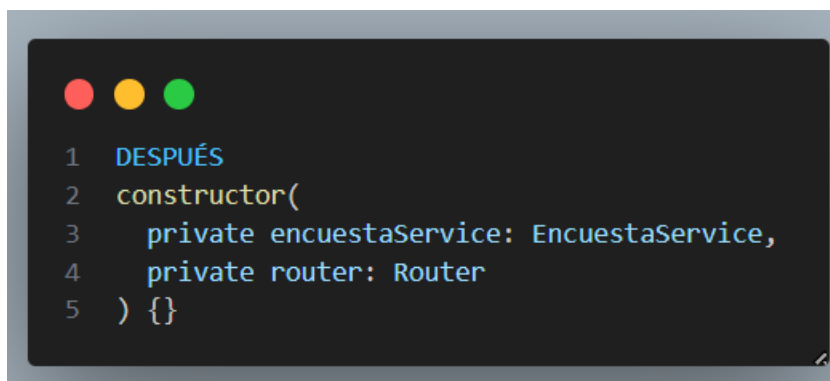
La figura muestra la implementación completa del archivo encuesta.service.ts, donde se centralizan todas las operaciones HTTP relacionadas con la entidad Encuesta. El servicio consume la URL base definida en environment.apiUrl y expone métodos específicos (crear, obtenerTodas, obtenerPorId, votar) que encapsulan la comunicación con el backend.

Paso 3 – Refactorizar crear.component.ts



```
1  ANTES
2  constructor(private http: HttpClient, private router: Router) {}
3  this.http.post(this.url + '/crear', { pregunta: this.pregunta })
```

Ilustración 50- Estado inicial de crear.component.ts con HttpClient directo (antes de la refactorización).

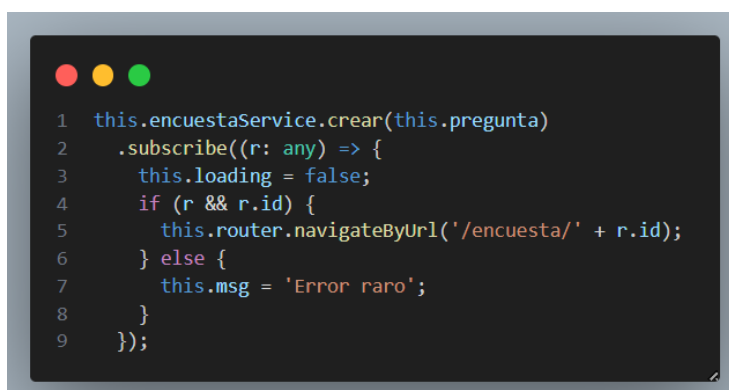


```
1  DESPUÉS
2  constructor(
3    private encuestaService: EncuestaService,
4    private router: Router
5  ) {}
```

Ilustración 51-crear.component.ts refactorizado consumiendo EncuestaService.

La figura muestra el estado posterior a la refactorización del componente crear.component.ts.

En esta versión, el HttpClient deja de inyectarse directamente en el componente y es reemplazado por EncuestaService, delegando la comunicación con el backend a la capa de servicio.

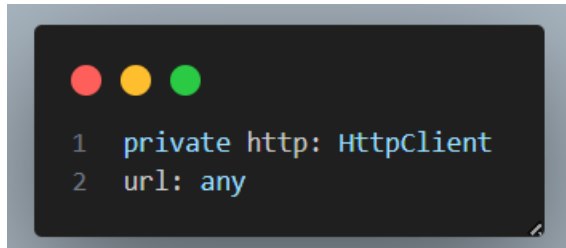


```
1  this.encuestaService.crear(this.pregunta)
2    .subscribe((r: any) => {
3    this.loading = false;
4    if (r && r.id) {
5      this.router.navigateByUrl('/encuesta/' + r.id);
6    } else {
7      this.msg = 'Error raro';
8    }
9  });
```

Ilustración 52- Consumo de EncuestaService desde crear.component.ts (estado refactorizado).

La figura muestra cómo el componente delega la operación de creación de encuestas al método `crear()` del `EncuestaService`. El componente únicamente se encarga de gestionar el estado de la interfaz (loading, navegación y mensajes), mientras que la lógica de comunicación con la API permanece encapsulada en la capa de servicio.

Eliminar completamente:



```
1 private http: HttpClient
2 url: any
```

Ilustración 53-Eliminación de dependencias directas a HttpClient y URL hardcodeada en el componente.

La eliminación de estas dependencias consolida la transición hacia una arquitectura más limpia, donde los componentes no gestionan detalles de comunicación HTTP ni configuración de entorno

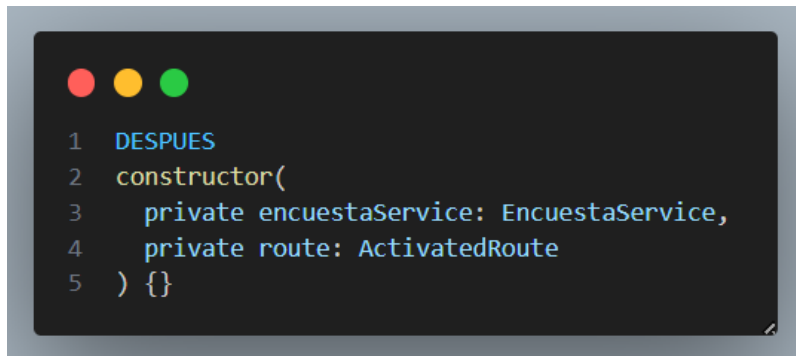
Paso 4- Refactorizar `encuesta.component.ts`



```
1 ANTES
2 constructor(private http: HttpClient, private route: ActivatedRoute) {}
3 this.http.get(this.url + '/encuesta/' + this.id)
4 this.http.post(this.url + '/votar', { id: this.id, voto: v })
```

Ilustración 54-Estado inicial de encuesta.component.ts con acceso HTTP directo (antes de la refactorización).

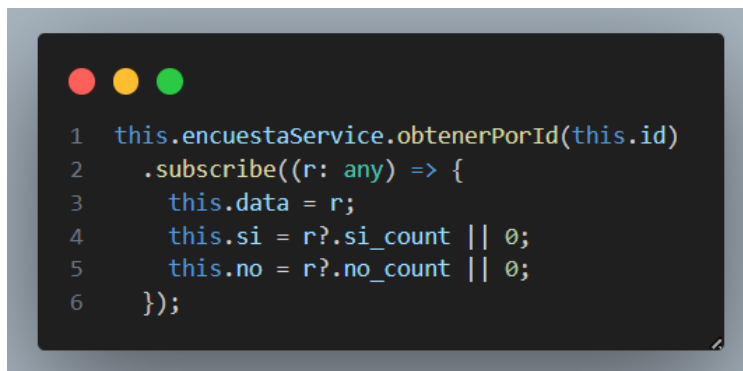
En su estado inicial, el componente asumía responsabilidades de comunicación con la API, lo cual generaba acoplamiento innecesario y duplicación de lógica. Esta situación justificó la aplicación de la Service Layer también en este módulo.



```
1  DESPUES
2  constructor(
3    private encuestaService: EncuestaService,
4    private route: ActivatedRoute
5  ) {}
```

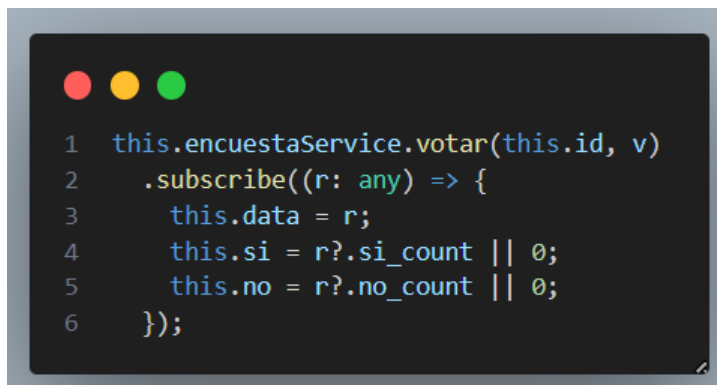
Ilustración 55-encuesta.component.ts refactorizado utilizando EncuestaService

La refactorización de encuesta.component.ts consolida la adopción de la Service Layer en el frontend, garantizando que la capa de presentación se limite exclusivamente a la gestión del estado y la interacción del usuario.



```
1  this.encuestaService.obtenerPorId(this.id)
2    .subscribe((r: any) => {
3    this.data = r;
4    this.si = r?.si_count || 0;
5    this.no = r?.no_count || 0;
6  });
```

Ilustración 56-Consumo del método obtenerPorId() desde EncuestaService.



```
1  this.encuestaService.votar(this.id, v)
2    .subscribe((r: any) => {
3    this.data = r;
4    this.si = r?.si_count || 0;
5    this.no = r?.no_count || 0;
6  });
```

Ilustración 57-Delegación del registro de voto al EncuestaService.

Con la delegación de las operaciones de consulta y votación al servicio, encuesta.component.ts queda completamente desacoplado de la infraestructura HTTP, consolidando una arquitectura frontend más limpia y mantenible.

Paso 5- Refactorizar respuestas.component.ts

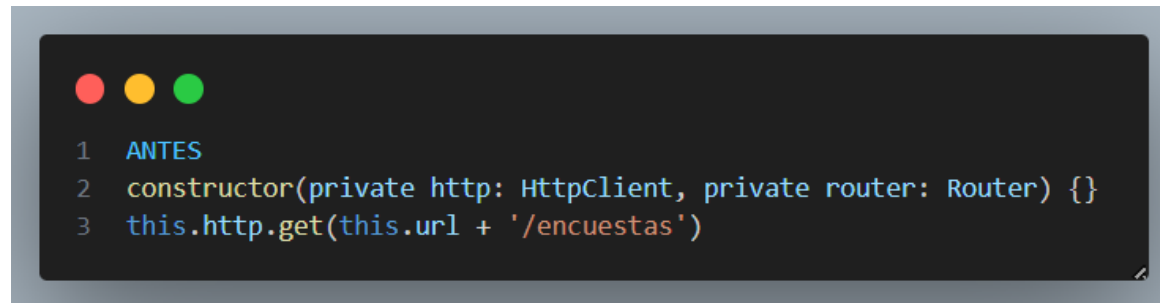


Ilustración 58-Estado inicial de respuestas.component.ts con acceso HTTP directo.

La implementación inicial del componente replicaba el anti-patrón de acceso directo a la infraestructura, justificando la necesidad de aplicar la Service Layer también en este módulo.

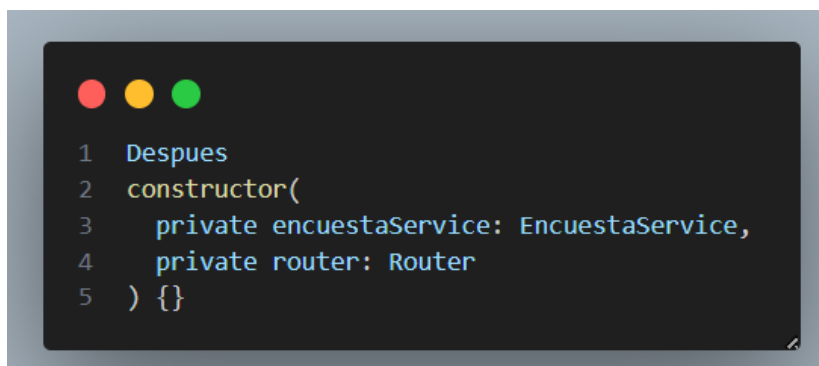
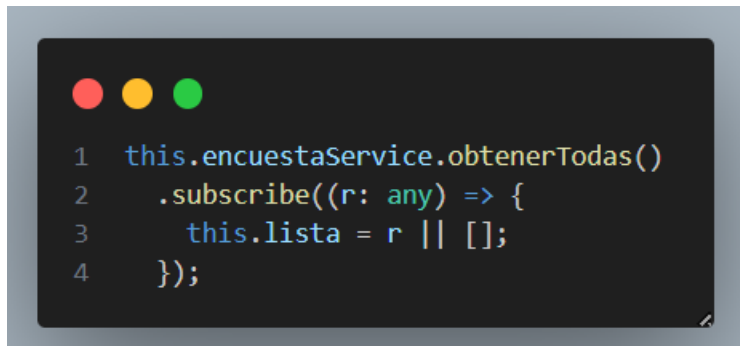


Ilustración 59-respuestas.component.ts refactorizado utilizando EncuestaService.

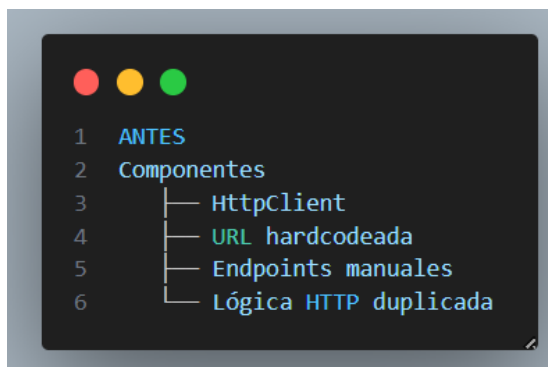
Con la refactorización de respuestas.component.ts, todos los componentes del frontend quedan completamente desacoplados de la infraestructura HTTP, consolidando la implementación del patrón Service Layer en Angular.



```
1 this.encuestaService.obtenerTodas()
2   .subscribe((r: any) => {
3     this.lista = r || [];
4   });
```

Ilustración 60-Consumo del método obtenerTodas() desde EncuestaService.

Cambio Estructural



```
1 ANTES
2 Componentes
3   ├── HttpClient
4   ├── URL hardcodeada
5   ├── Endpoints manuales
6   └── Lógica HTTP duplicada
```

Ilustración 61-Estructura inicial del frontend antes de la refactorización.



```
1 Después
2 Componentes
3   ↓
4 EncuestaService
5   ↓
6 HttpClient
7   ↓
8 Backend API
```

Ilustración 62- Estructura del frontend después de implementar la Service Layer.

La figura muestra la arquitectura del frontend tras la refactorización. En esta nueva estructura, los componentes ya no gestionan directamente la comunicación HTTP, sino que delegan dicha

responsabilidad al EncuestaService. Este servicio centraliza las llamadas a la API utilizando HttpClient, el cual finalmente se comunica con el backend. Esta organización mejora la modularidad, reduce el acoplamiento y facilita el mantenimiento del sistema.

4. Actividad 3.2 – Propuesta de Principios Clean Code

Tras la aplicación de patrones arquitectónicos en la Actividad 3.1, el siguiente paso consiste en mejorar la calidad interna del código mediante la aplicación de principios Clean Code.

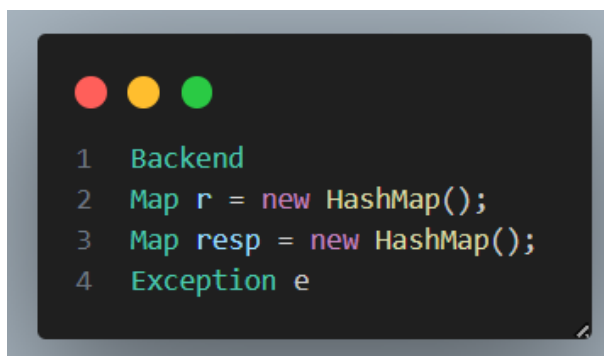
Mientras que los patrones estructurales abordan problemas de arquitectura, Clean Code se enfoca en:

- Claridad
- Legibilidad
- Mantenibilidad
- Reducción de complejidad
- Eliminación de duplicación
- Mejora semántica

El sistema original presenta múltiples violaciones a estos principios tanto en backend como en frontend.

4.1 Nombres Significativos

En la versión inicial del sistema se utilizan nombres poco descriptivos como:

A screenshot of a code editor with a dark background and light-colored text. The code is written in Java and shows four lines of code. The first line is a comment: `1 Backend`. The second line is `2 Map r = new HashMap();`. The third line is `3 Map resp = new HashMap();`. The fourth line is `4 Exception e`. The code is highlighted with a light blue selection box.

```
1 Backend
2 Map r = new HashMap();
3 Map resp = new HashMap();
4 Exception e
```

Ilustración 63- Uso de nombres genéricos en el backend (antes de aplicar Clean Code).



Ilustración 64-Usos de nombres genéricos en el frontend (antes de aplicar Clean Code).

Este tipo de nomenclatura:

- No comunica intención.
- Reduce la claridad.
- Aumenta la carga cognitiva.
- Dificulta mantenimiento

Propuesta de Mejora

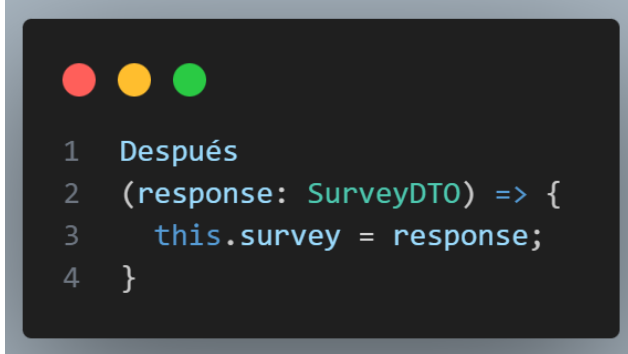
Renombrar variables y objetos según su intención semántica.



Ilustración 65-Variables renombradas según su intención semántica (Clean Code aplicado).

La figura muestra la mejora aplicada en el backend al renombrar variables genéricas como `r` o `resp` por identificadores más descriptivos como `response`. Además, se especifica el tipo de datos utilizando `Map<String, Object>`, lo que mejora la claridad del código y reduce ambigüedad.

Frontend:



```
1  Después
2  (response: SurveyDTO) => {
3      this.survey = response;
4  }
```

Ilustración 66- Uso de nombres descriptivos y tipado explícito en el frontend.

la mejora aplicada en el frontend al reemplazar variables genéricas como `r` por identificadores descriptivos como `response`, además de tiparlas explícitamente como `SurveyDTO`. Este cambio mejora la claridad semántica del código y aprovecha el sistema de tipos de TypeScript para reforzar la seguridad y legibilidad.

Beneficios

- Código autoexplicativo.
- Menor necesidad de comentarios.
- Mejor legibilidad.
- Mayor claridad semántica.

La aplicación del principio de Nombres Significativos mejora considerablemente la legibilidad y mantenibilidad del sistema tanto en backend como en frontend. Al utilizar identificadores descriptivos y tipos explícitos, el código se vuelve más claro, seguro y alineado con buenas prácticas de ingeniería de software.

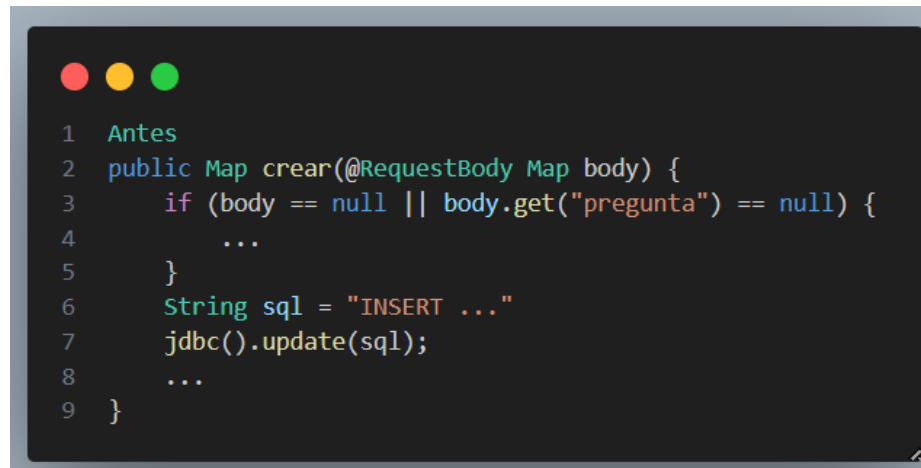
4.2 Funciones Pequeñas y Responsabilidad Única

El método `crear()` en el backend concentra múltiples responsabilidades:

- Validación
- Construcción SQL

- Persistencia
- Construcción de respuesta
- Manejo de errores

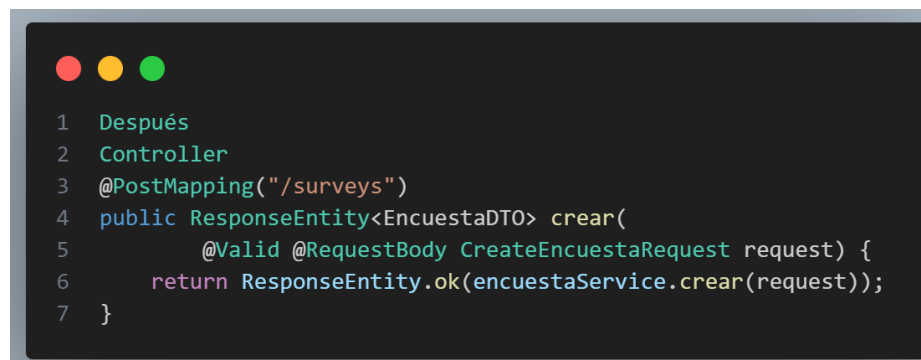
Esto viola SRP y dificulta testing.



```
1  Antes
2  public Map crear(@RequestBody Map body) {
3      if (body == null || body.get("pregunta") == null) {
4          ...
5      }
6      String sql = "INSERT ...";
7      jdbc().update(sql);
8      ...
9  }
```

Ilustración 67-Función con múltiples responsabilidades (violación del principio de responsabilidad única).

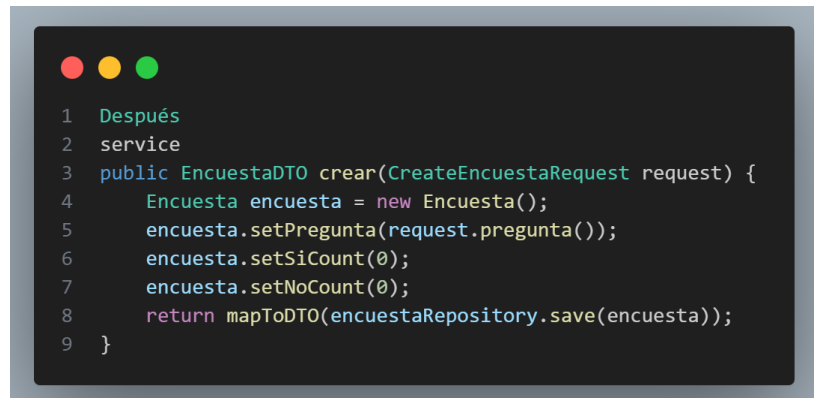
Separación en capas:



```
1  Después
2  Controller
3  @PostMapping("/surveys")
4  public ResponseEntity<EncuestaDTO> crear(
5      @Valid @RequestBody CreateEncuestaRequest request) {
6      return ResponseEntity.ok(encuestaService.crear(request));
7  }
```

Ilustración 68-Refactorización del Controller bajo el principio de Responsabilidad Única.

Service:

A screenshot of a code editor with a dark background and light-colored text. At the top left, there are three colored circles: red, yellow, and green. The code is written in Java and shows a service method named 'crear' that takes a 'CreateEncuestaRequest' object as a parameter. The method creates a new 'Encuesta' object, sets its 'pregunta' property, sets 'SiCount' and 'NoCount' to 0, and then returns the result of 'mapToDTO(encuestaRepository.save(encuesta))'.

```
1 Después
2 service
3 public EncuestaDTO crear(CreateEncuestaRequest request) {
4     Encuesta encuesta = new Encuesta();
5     encuesta.setPregunta(request.pregunta());
6     encuesta.setSiCount(0);
7     encuesta.setNoCount(0);
8     return mapToDTO(encuestaRepository.save(encuesta));
9 }
```

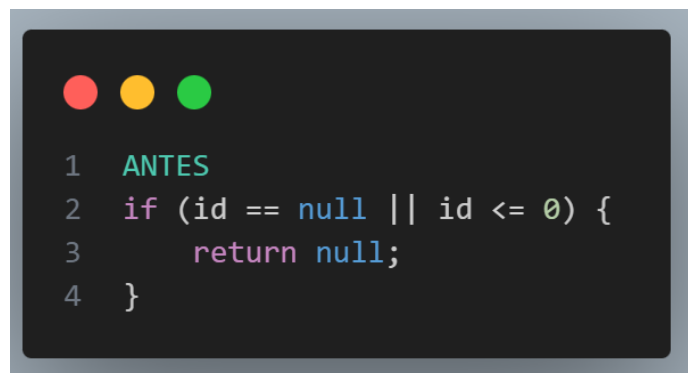
Ilustración 69-Aplicación del principio de Responsabilidad Única en la capa Service.

Beneficios

- Métodos pequeños.
- Más testeable.
- Mayor claridad.
- Mejor separación de responsabilidades.

4.3 DRY (Don't Repeat Yourself)

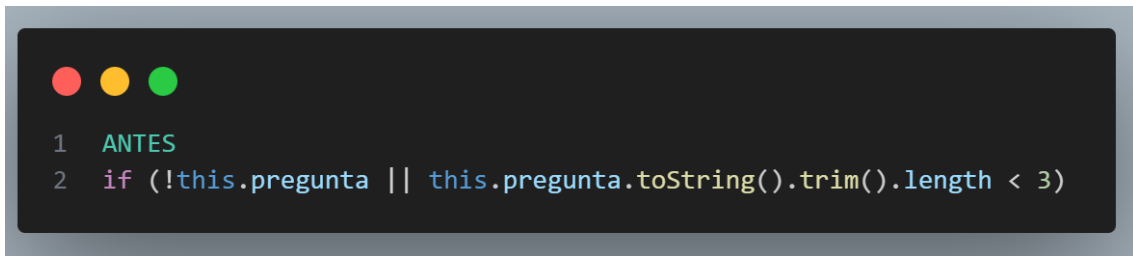
Validaciones repetidas en múltiples métodos:

A screenshot of a code editor with a dark background and light-colored text. At the top left, there are three colored circles: red, yellow, and green. The code is written in Java and shows a validation check for an 'id' parameter. It uses an 'if' statement to check if 'id' is null or less than or equal to 0. If true, it returns null. The code is labeled 'ANTES' at the top.

```
1 ANTES
2 if (id == null || id <= 0) {
3     return null;
4 }
```

Ilustración 70-Aplicación del principio DRY: Eliminación de validaciones repetidas en el código.

Y en frontend:

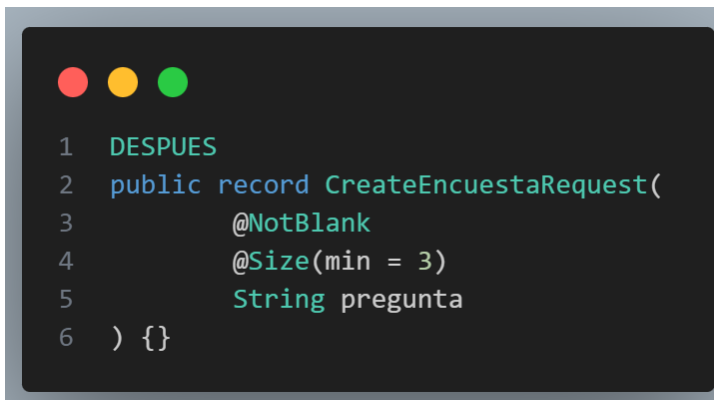


```
1 ANTES
2 if (!this.pregunta || this.pregunta.toString().trim().length < 3)
```

Ilustración 71-Aplicación del principio DRY en el Frontend: Centralización y reutilización de validaciones de entrada.

Propuesta

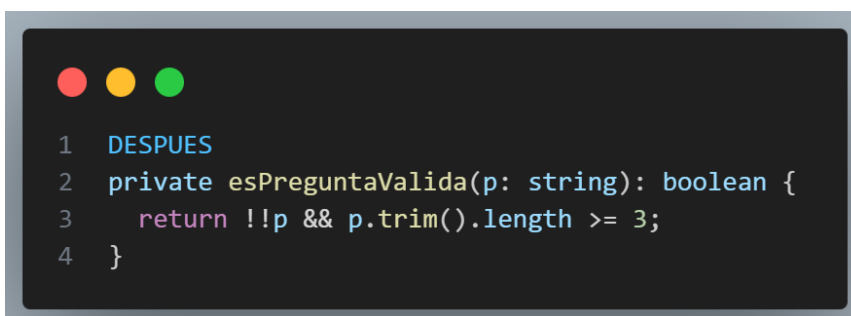
Backend → Validación declarativa:



```
1 DESPUES
2 public record CreateEncuestaRequest(
3     @NotBlank
4     @Size(min = 3)
5     String pregunta
6 ) {}
```

Ilustración 72-Aplicación de validación declarativa en el Backend para eliminar validaciones repetidas (DRY).

Frontend → Centralizar validación en método privado:



```
1 DESPUES
2 private esPreguntaValida(p: string): boolean {
3     return !!p && p.trim().length >= 3;
4 }
```

Ilustración 73-Refactorización de validaciones en el Frontend aplicando el principio DRY.

Beneficios

- Eliminación de duplicación.
- Mayor consistencia.
- Código más limpio.

4.4 Manejo de Errores – Evitar return null



```
1  ANTES
2  catch (Exception e) {
3      e.printStackTrace();
4  }
5  return null;
```

Ilustración 74-Manejo inadecuado de excepciones: uso de printStackTrace() y retorno de null como anti-patrón.

Esto genera:

- Respuestas inconsistentes.
- NullPointerException potencial.
- API poco profesional.

Propuesta

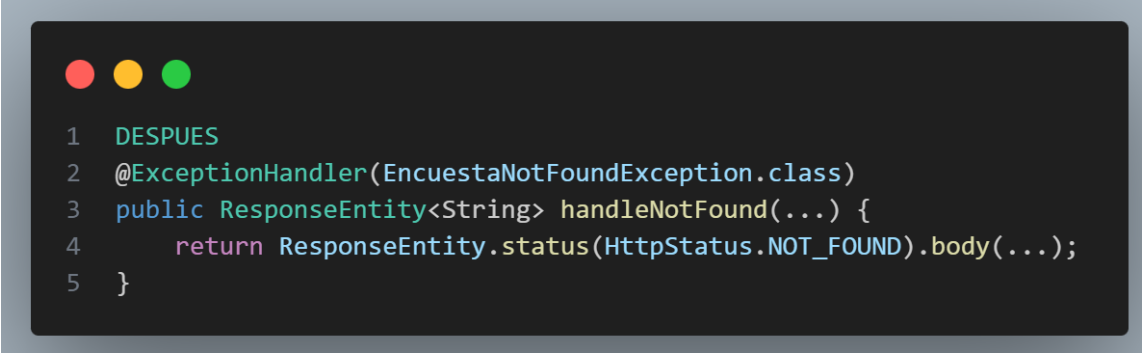
Excepción personalizada:



```
1  DESPUES
2  public class EncuestaNotFoundException extends RuntimeException {
3      public EncuestaNotFoundException(Long id) {
4          super("Encuesta no encontrada: " + id);
5      }
6  }
```

Ilustración 75- Implementación de excepción personalizada para manejo semántico de errores.

Manejo centralizado:



```
1  DESPUES
2  @ExceptionHandler(EncuestaNotFoundException.class)
3  public ResponseEntity<String> handleNotFound(...) {
4      return ResponseEntity.status(HttpStatus.NOT_FOUND).body(...);
5  }
```

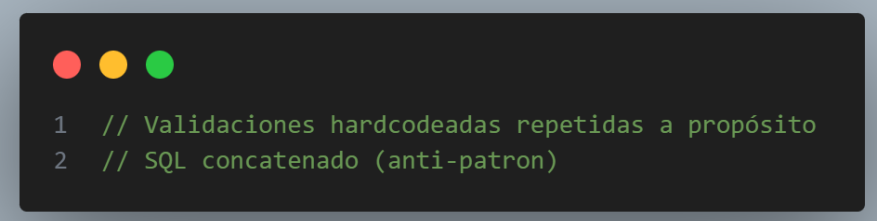
Ilustración 76- Respuesta HTTP semántica mediante manejo global de excepciones.

Beneficios

- API consistente.
- No retorna null.
- Claridad en flujo de errores.

4.5 Comentarios y Código Autoexplicativo

Comentarios redundantes:



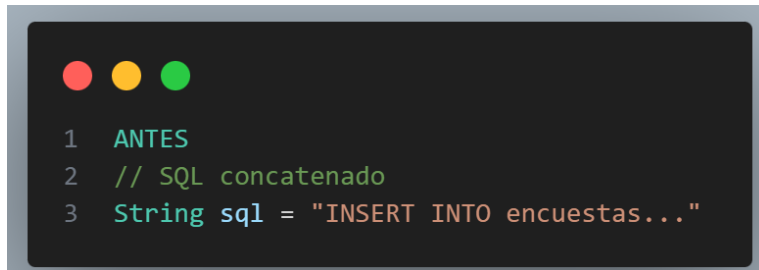
```
1  // Validaciones hardcodeadas repetidas a propósito
2  // SQL concatenado (anti-patron)
```

Ilustración 77-Eliminación de comentarios innecesarios mediante código autoexplicativo.

Clean Code propone: el código debe explicar lo que hace.

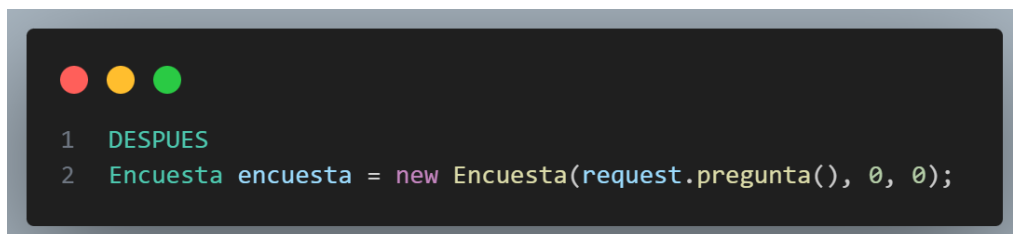
Propuesta

Eliminar comentarios innecesarios y mejorar nombres.



```
1  ANTES
2  // SQL concatenado
3  String sql = "INSERT INTO encuestas..."
```

Ilustración 78- Uso de SQL concatenado como anti-patrón y código no autoexplicativo.



```
1  DESPUES
2  Encuesta encuesta = new Encuesta(request.pregunta(), 0, 0);
```

Ilustración 79- Código autoexplicativo mediante inicialización clara de la entidad.

La entidad se construye directamente con valores iniciales definidos, eliminando comentarios innecesarios y mejorando la claridad e intención del código.

Impacto General

La aplicación de principios Clean Code permite:

- Reducción de complejidad.
- Mejor legibilidad.
- Eliminación de código duplicado.
- Mejor manejo de errores.
- Mayor mantenibilidad.
- Base sólida para microservicios.

Conexión con Microservicios

Clean Code es fundamental para:

- Descomponer el sistema.
- Definir bounded contexts claros.
- Evitar deuda técnica distribuida.
- Facilitar testing en entornos independientes.

Conclusión

La aplicación sistemática de principios Clean Code transforma el sistema de una implementación funcional pero desorganizada a una solución estructurada, legible y preparada para evolución arquitectónica.

5. Actividad 3.3 – Diseño de Arquitectura de Microservicios

En las fases anteriores se identificó que el sistema “Espagueti de Encuestas” opera como un monolito acoplado, donde:

- El backend concentra múltiples responsabilidades.
- No existe separación clara de dominios.
- El acceso a datos y la lógica de negocio están interconectados.
- El sistema no puede escalar de manera independiente.
- No existe autonomía por módulo funcional.

Aunque en la Actividad 3.1 se aplicó una arquitectura por capas, el sistema continúa siendo un monolito desplegable como una sola unidad.

Por esta razón, se propone una evolución hacia una arquitectura basada en microservicios.

5.1 Identificación de Bounded Contexts

El primer paso en la descomposición es identificar dominios funcionales independientes (Bounded Contexts). Analizando el sistema actual, se detectan dos dominios claramente diferenciados:

- Gestión de encuestas (creación y consulta) y Registro y conteo de votos.

Estos dominios poseen responsabilidades distintas y diferentes características de carga y escalabilidad.

5.2 Propuesta de Microservicios

Se propone dividir el monolito en los siguientes microservicios:

5.3.1 survey-service

Responsabilidad

- Crear encuestas.
- Consultar encuestas.
- Gestionar ciclo de vida de encuestas.

Endpoints

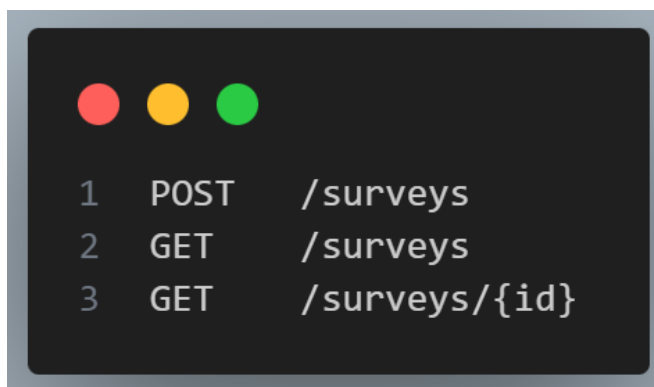


Ilustración 80-Diseño del microservicio survey-service y definición de endpoints REST.

Base de datos

Base de datos propia:

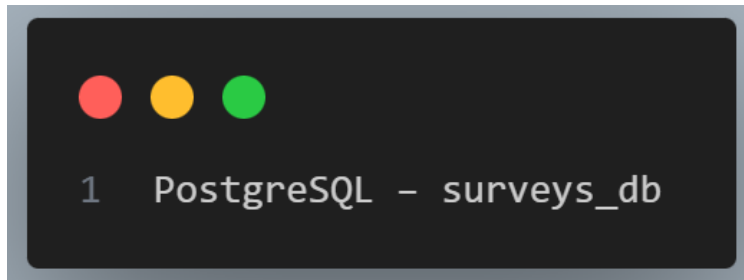


Ilustración 81- onfiguración de base de datos PostgreSQL – surveys_db.

Este servicio no conoce votos individuales.

5.3.2 voting-service

Responsabilidad

- Registrar votos.
- Calcular resultados.
- Gestionar conteo.

Endpoints

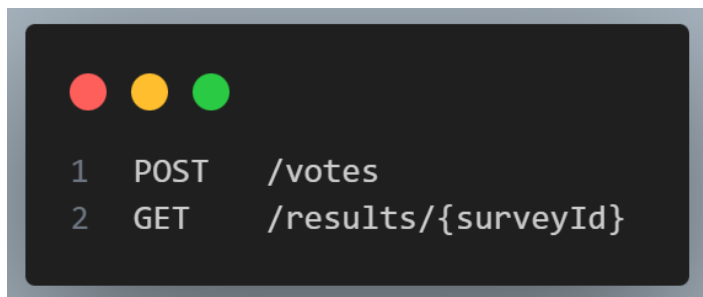


Ilustración 82-Diseño del microservicio voting-service y definición de endpoints.

Base de datos

Base de datos independiente:

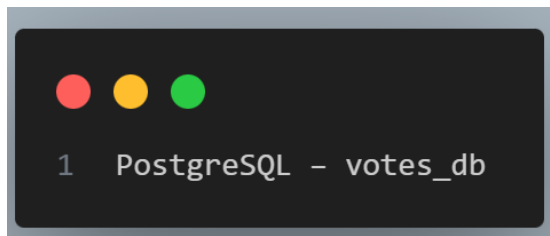


Ilustración 83-Configuración de base de datos del voting-service – votes_db.

Este servicio no accede directamente a la base de datos del survey-service.

5.3.3 api-gateway

Responsabilidad

- Punto único de entrada.
- Autenticación JWT.
- *Rate limiting*.
- Enrutamiento a microservicios.

Ejemplo de enrutamiento:

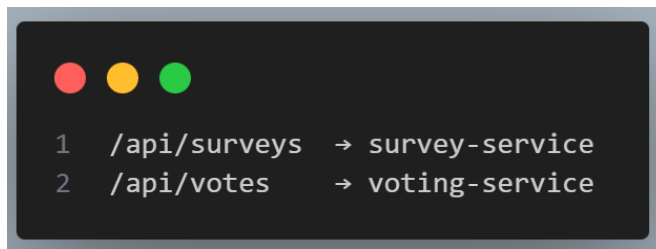


Ilustración 84-Implementación del api-gateway y enrutamiento hacia microservicios.

El frontend nunca llama directamente a los microservicios.

5.4 Arquitectura Propuesta

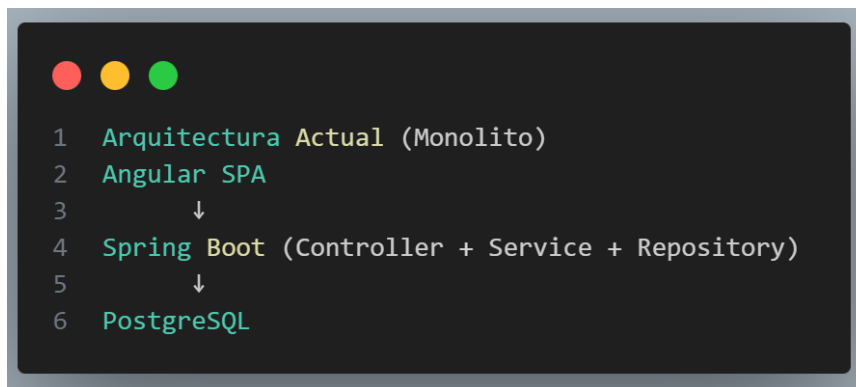


Ilustración 85-Arquitectura Actual del Sistema – Enfoque Monolítico.

Limitaciones:

- Escalado completo.
- Despliegue único.
- Alta dependencia interna.

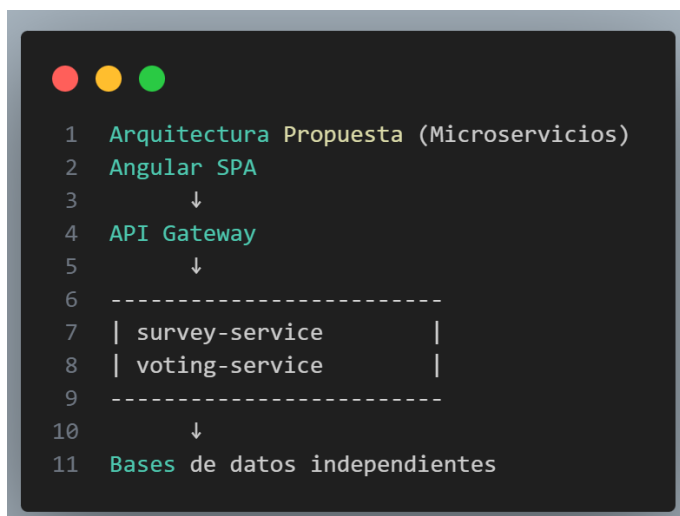


Ilustración 86-Arquitectura Propuesta basada en Microservicios.

Cada servicio:

- Se despliega independientemente.
- Escala de forma autónoma.
- Posee su propia base de datos.

5.5 Componentes de Infraestructura Adicionales

Para soportar una arquitectura de microservicios robusta, se proponen los siguientes componentes:

5.5.1 Service Discovery (Eureka / Consul)

Permite:

- Registro dinámico de instancias.
- Balanceo automático.
- Tolerancia a fallos.

5.5.2 Config Server Centralizado

Permite:

- Externalizar configuración.
- Gestionar múltiples ambientes.
- Centralizar application.yml.

5.5.3 Circuit Breaker (Resilience4j)

Permite:

- Evitar fallos en cascada.
- Controlar servicios no disponibles.
- Implementar fallback.

5.5.4 Centralized Logging (ELK Stack)

Permite:

- Monitoreo centralizado.
- Trazabilidad.
- Observabilidad distribuida.

5.6 Beneficios de la Descomposición

La migración a microservicios aporta:

- Escalabilidad independiente.
- Mejor tolerancia a fallos.
- Equipos trabajando en paralelo.
- Evolución independiente de dominios.
- Mayor resiliencia.
- Mejor observabilidad.
- Preparación para cloud-native.

5.7 Conexión con las Fases Anteriores

La arquitectura por capas aplicada en 3.1 facilita la transición hacia microservicios.

Los DTOs definidos en 3.1 se convierten en contratos de servicio.

La externalización de configuración facilita despliegues independientes.

El Global Exception Handler estandariza respuestas.

La Service Layer en Angular permite consumir múltiples servicios sin modificar componentes.

5.8 conclusión de la Actividad 3.3

La propuesta de microservicios representa una evolución estratégica del sistema “Espagueti de Encuestas”, pasando de un monolito acoplado a una arquitectura distribuida basada en dominios independientes.

Esta transformación no solo mejora la escalabilidad y resiliencia del sistema, sino que establece una base sólida para su crecimiento futuro y adaptación a entornos cloud y arquitecturas modernas.

ADR-001 · Adoptar arquitectura de microservicios

Estado: ACCEPTED

Contexto

El sistema actual funciona como un monolito donde se mezclan acceso a datos, lógica de negocio y presentación en EncuestaController.java.

La funcionalidad de votación presenta picos de carga distintos a la creación de encuestas, lo que dificulta el escalado independiente.

Decisión

Separar el sistema en dos microservicios:

survey-service (gestión de encuestas)

voting-service (registro y conteo de votos)

Ambos serán consumidos a través de un API Gateway.

Consecuencias

Escalado independiente.

Mejor separación de responsabilidades.

pueden trabajar en paralelo.

Mayor complejidad operativa.

Necesidad de Service Discovery y Config Server.

ADR-002 · Usar Spring Data JPA en lugar de JDBC manual

Estado: ACCEPTED

Contexto

El sistema utiliza JdbcTemplate con SQL concatenado directamente en el controlador, lo que genera:

Riesgo de SQL Injection.

Acoplamiento entre web y base de datos.

Código repetitivo y poco mantenible.

Decisión

Adoptar Spring Data JPA con:

Entidades (@Entity)

Repositorios (JpaRepository)

Eliminación de JdbcTemplate manual

Consecuencias

Eliminación de SQL Injection.

Separación clara de capas.

Código más limpio y mantenible.

Mejor preparación para microservicios.

Dependencia de ORM (Hibernate).

Curva de aprendizaje inicial

Conclusión General

Durante la Fase 3 se abordó la modernización integral del sistema desde tres dimensiones: patrones de diseño, principios Clean Code y arquitectura de microservicios.

En la Actividad 3.1, se identificaron anti-patrones (código espagueti, responsabilidades mezcladas, lógica duplicada) y se propusieron soluciones basadas en patrones como Layered Architecture, Service Layer, Repository y DTO, logrando una estructura más organizada y mantenible.

En la Actividad 3.2, se aplicaron principios de Clean Code:

- Nombres significativos.
- Funciones pequeñas y responsabilidad única.
- DRY (eliminación de duplicación).
- Manejo adecuado y centralizado de excepciones.

Esto permitió mejorar la claridad, cohesión y calidad semántica del código tanto en backend como en frontend.

En la Actividad 3.3, se propuso la evolución arquitectónica hacia microservicios, separando el sistema en servicios independientes (survey-service y voting-service), con API Gateway y bases de datos aisladas, mejorando escalabilidad y desacoplamiento.